



**MUHAMMAD NAWAZ SHARIF
UNIVERSITY OF ENGINEERING
AND TECHNOLOGY, MULTAN**

LAB Manual

HCI and Computer Graphics

Name

M.JUNAID

Reg. No

2023-IT-39

Degree

BS-IT

Session

2023-2027

Submitted to

MR MOIL WASEEM

LAB NO#01

Figma Overview (Layer Panel, Design/Properties Panel, Tools) + Login Page Design

Part 1: Figma Interface Overview

Figma is a powerful, cloud-based collaborative design tool widely used for UI/UX design. In this lab, we explored its fundamental interface components.

1. Layers Panel (Left Sidebar)

The Layers Panel is located on the left side of the Figma interface. It displays all the design elements in a hierarchical and organized list, making it easy to manage complex designs.

Key Functions:

Displays all elements such as frames, text layers, images, vectors, and shapes.

Allows quick selection of specific objects.

Supports renaming, grouping, and ungrouping of layers.

Controls the stacking order of layers (bringing elements forward or sending them backward).

Example Layer Structure (Login Page):

- Frame (Login Page)
- Rectangle (Background)
- Text (Login Heading)
- Rectangle (Email Input Field)
- Rectangle (Password Input Field)
- Rectangle (Login Button)
- Text (Forgot Password?)
- Text (Don't have an account? Sign up)

2. Design / Properties Panel (Right Sidebar)

The Design Panel appears on the right side of the screen when an element is selected. It provides complete control over the visual properties of the selected object.

Key Functions:

- Modify width and height of elements.
- Change fill colors, stroke (border) colors, and opacity.

- Adjust text properties including font family, size, weight, alignment, and line height.
- Add and customize effects such as shadows, blurs, and corner radius.

3. Tools Panel (Top Toolbar)

The Tools Panel is located at the top of the Figma workspace and contains all the essential drawing and editing tools.

Commonly Used Tools:

- **Move Tool (V)** – Select and move objects around the canvas.
- **Frame Tool (F)** – Create artboards or screen frames.
- **Rectangle Tool (R)** – Create rectangles, buttons, and input fields.
- **Text Tool (T)** – Add and edit text layers.
- **Line and Shape Tools** – Draw lines, ellipses, polygons, stars, and other vector shapes.

Part 2: Practical Task – Login Page Design

Objective:

Design a clean and user-friendly login page using Figma.

Step-by-Step Procedure:

Step 1: Create a New Frame

- Open Figma and create a new file.
- Select the Frame Tool (F) from the toolbar.
- Choose a preset size: Desktop (1440 × 1024 px) or Mobile (375 × 812 px).

Step 2: Add Background

- Draw a rectangle using the Rectangle Tool (R) to cover the entire frame.
- Set a light background color (e.g., white or soft light blue).

Step 3: Add Login Heading

- Select the Text Tool (T).
- Type “Login”.
- Set font size between 24–32 px, make it bold, and center-align it.

Step 4: Create Input Fields

- Draw two rectangles for Email and Password fields.
- Set height: 40–45 px.
- Apply a light gray border.
- Add placeholder text inside each field (e.g., “Enter your email”, “Enter your password”).

Step 5: Design Login Button

- Create a rectangle for the button.
- Fill it with a prominent blue color.
- Add the text “Login” in white color.
- Center-align the text and apply a corner radius of 6–8 px for a modern look.

Step 6: Add Extra Text Links

- Add “Forgot Password?” text.
- Add “Don’t have an account? Sign up” text at the bottom.

Components Used in Login Page:

- Frame
- Rectangle
- Text
- Alignment Tools
- Color Styles
- Layer Management

HCI Concepts Applied:

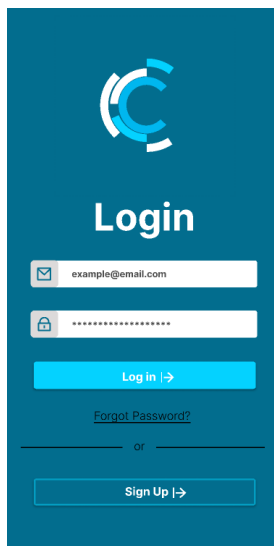
Usability: Simple and intuitive layout for easy navigation.

Consistency: Uniform spacing, colors, and typography throughout the design.

Visibility: Clear text and prominent interactive elements.

User-Friendly Interface: Minimal cognitive load with a clean design.

Login Page:



Conclusion:

In this lab, we familiarized ourselves with the core components of the Figma interface - Layers Panel, Design/Properties Panel, and Tools Panel. We successfully designed a simple yet professional Login Page by applying fundamental design principles. This lab provided a strong foundation for understanding user interface design and user-centered design (UCD) concepts.

LAB NO#02

Constraints in Figma + Responsive Login Form + Shape Tools & Object Handling

Part 1: Understanding Constraints in Figma

What are Constraints?

Constraints in Figma are powerful features that control how design elements behave when their parent frame is resized. They play a crucial role in creating responsive designs that adapt gracefully to different screen sizes (desktop, tablet, and mobile).

Constraints help maintain proper alignment and proportions, ensuring a consistent user experience across devices.

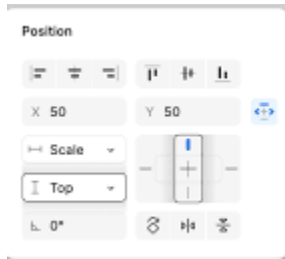
Types of Constraints

Horizontal Constraints:

- Left - Element stays fixed to the left side.
- Right - Element stays fixed to the right side.
- Center - Element remains horizontally centered.
- Left & Right - Element stretches or shrinks horizontally while maintaining distance from both sides.
- Scale - Element scales proportionally with the frame.

Vertical Constraints:

- Top - Element stays fixed to the top.
- Bottom - Element stays fixed to the bottom.
- Center - Element remains vertically centered.
- Top & Bottom - Element stretches vertically.
- Scale - Element scales proportionally with the frame.



Part 2: Applying Constraints to Create a Responsive Login Form

Objective:

Make the Login Page designed in Lab 01 responsive using Constraints.

Step-by-Step Application of Constraints:

1. Login Frame

- Select the main frame.
- Keep default constraints.

2. Login Heading (“Login”)

- Horizontal Constraint: Center
- Vertical Constraint: Top

→ The heading remains perfectly centered when the frame is resized.

3. Email & Password Input Fields

- Horizontal Constraint: Left & Right
- Vertical Constraint: Top

→ Input fields stretch horizontally while maintaining their position from the top.

4. Login Button

- Horizontal Constraint: Left & Right
- Vertical Constraint: Top

→ The button automatically adjusts its width when the frame size changes.

5. Extra Text (“Forgot Password?” and “Don’t have an account? Sign up”)

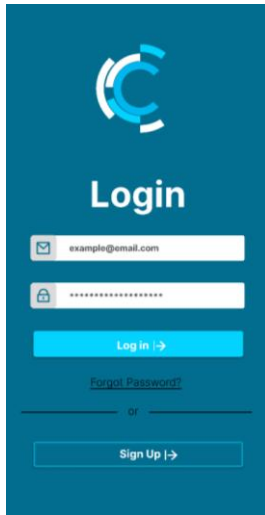
- Horizontal Constraint: Center
- Vertical Constraint: Bottom

→ Text remains centered at the bottom of the screen.

Result:

When the frame is resized:

- All elements stay properly aligned.
- The layout remains clean and user-friendly.
- The design becomes fully responsive.

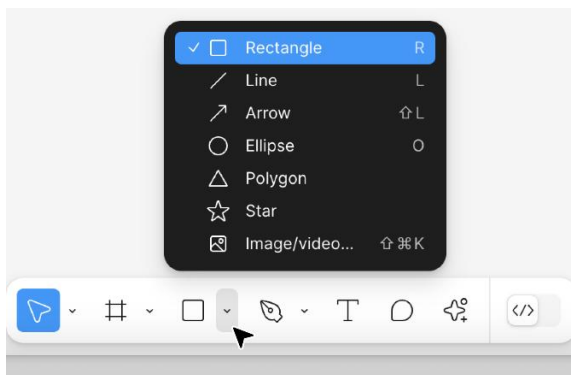


Part 3: Shape Tools in Figma

Figma provides various shape tools to build UI elements quickly.

Common Shape Tools:

- **Rectangle Tool (R)** - Used for buttons, input fields, cards, and backgrounds.
- **Ellipse Tool (O)** - Perfect for circular icons, profile pictures, and avatars.
- **Line Tool (L)** - Creates dividers and separators.
- **Polygon and Star Tools** - Used for decorative elements and icons.



Part 4: Working with Objects in Figma

Selection Techniques:

- Click on an object to select it.
- Shift + Click to select multiple objects.
- Drag a selection box around objects for group selection.

Copying and Duplicating Objects:

- Ctrl + C / Ctrl + V - Copy and paste.
- Alt + Drag - Duplicate while dragging.
- Ctrl + D - Repeat the last duplication.

Resizing Objects:

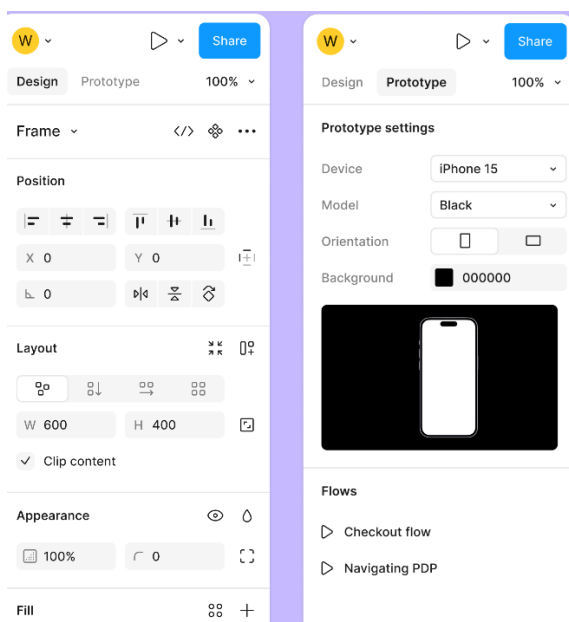
- Drag corner handles to resize.
- Hold Shift to maintain aspect ratio (proportional resizing).
- Enter exact values in the Design Panel for precision.

Grouping Objects:

Select multiple objects → Press Ctrl + G to group.

Groups make it easier to move and manage multiple elements together.

Ungroup: Ctrl + Shift + G.



Locking Layers:

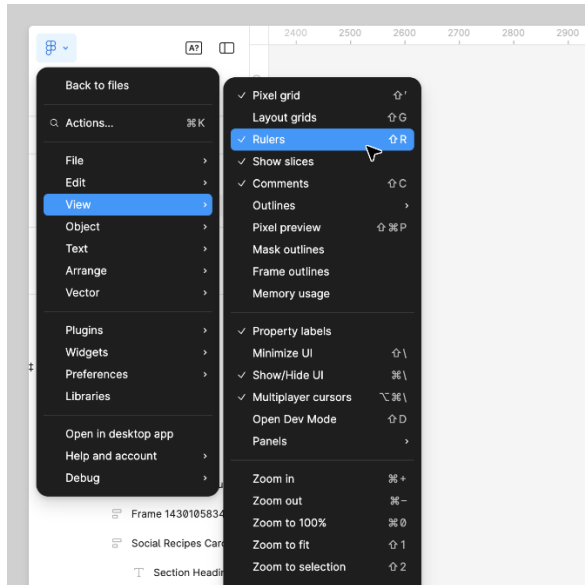
Select layer → Press Ctrl + Shift + L.

Prevents accidental movement or editing.

Rulers and Guides:

Enable rulers: Shift + R.

Drag guides from rulers for precise alignment.



Additional Adjustments:

- Change opacity.
- Apply corner radius for rounded corners.
- Add shadows and blurs.
- Modify stroke (border) thickness and style.

Spacing & Alignment:

- Use Smart Guides (auto spacing indicators).
- Alignment tools: Left, Center, Right, Top, Bottom, etc.
- Distribute objects with equal spacing for a clean and professional UI.

HCI Concepts Applied:

- **Responsiveness** - Designs adapt to different screen sizes.
- **Consistency** - Uniform layout and spacing across the interface.
- **Efficiency** - Grouping and constraints speed up the design process.
- **Error Prevention** - Locking layers avoids accidental changes.

Conclusion

In this lab, we learned how to use Constraints to create a responsive login form that adapts to different screen sizes. We also practiced essential shape tools and object handling techniques, including selection, duplication, grouping, locking, alignment, and guides. These skills are fundamental for building professional, scalable, and user-friendly interfaces in Figma.

LAB NO#03

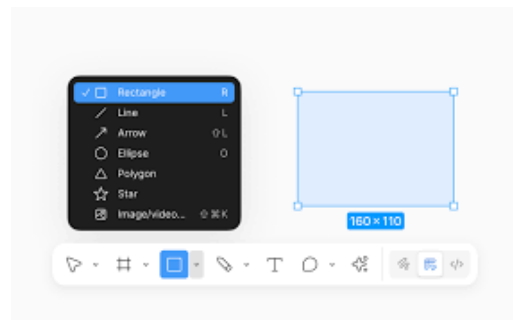
Shape Manipulation, Colors & Typography, Styles, and Wireframes

Part 1: Shape Manipulation in Figma

Shape manipulation refers to the process of editing and customizing basic shapes to create polished and functional UI components. It is a fundamental skill in UI/UX design.

Common Shape Manipulation Actions:

- Resize shapes by dragging corner handles or entering precise values in the Design Panel.
- Rotate shapes to different angles.
- Adjust corner radius to create rounded edges.
- Duplicate and align shapes for consistency.



Fill

Fill determines the interior color or appearance of a shape.

Types of Fills:

- Solid Color
- Linear Gradient
- Radial Gradient
- Image Fill

Use Cases: Buttons, cards, backgrounds, and containers.

Stroke

A stroke defines the border or outline of a shape.

Options:

- Stroke Color
- Stroke Width (thickness)
- Stroke Position (Inside, Outside, or Center)
- Dashed or Dotted borders

Use Cases: Input fields, outlines, icons, and dividers.

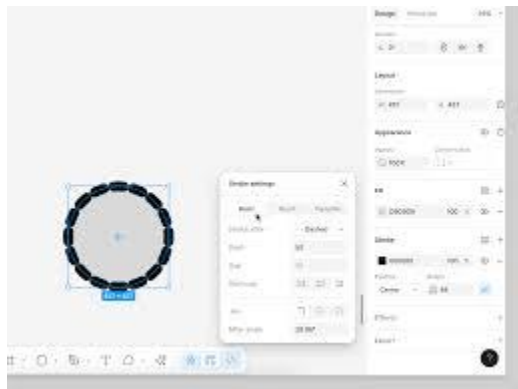
Effects

Effects are used to enhance the visual depth and appeal of elements.

Types of Effects:

- Drop Shadow
- Inner Shadow
- Layer Blur
- Background Blur

Use Cases: Cards, modals, floating action buttons, and highlights.



Boolean Operations

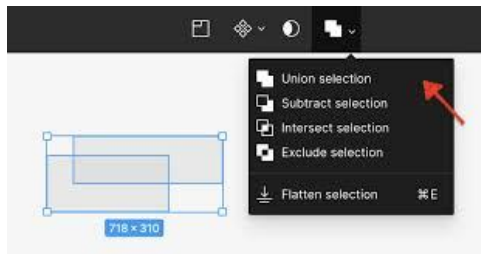
Boolean operations allow designers to combine or modify multiple shapes to create complex custom designs.

Types:

- **Union** - Merges multiple shapes into one.

- **Subtract** - Removes the overlapping area of one shape from another.
- **Intersect** - Keeps only the overlapping area.
- **Exclude** - Removes the intersection area.

Use Cases: Creating custom icons, logos, and complex symbols.



Part 2: Color Theory and Typography in UI Design

Color Theory in UI

Colors play a vital role in guiding user attention, establishing visual hierarchy, and conveying meaning in an interface.

Common Color Types:

- **Primary Color** - Used for main actions (e.g., buttons, links).
- **Secondary Color** - Supports primary actions.
- **Background Color** - Main canvas color.
- **Accent Color** - Highlights important elements.

Best Practices:

Maintain high contrast for better readability.

Use a limited color palette (usually 3–5 colors) for visual harmony.

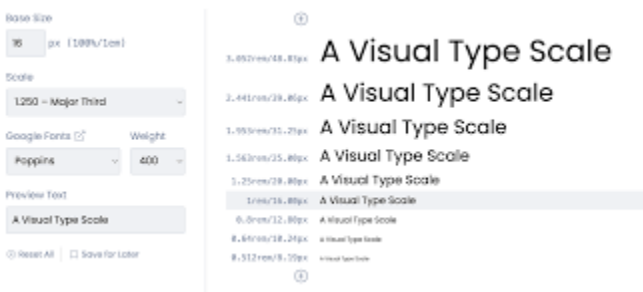


Typography

Typography is the art of arranging and styling text to make it readable and visually appealing.

Important Typography Properties:

- Font Family
- Font Size
- Line Height
- Letter Spacing
- Font Weight (Light, Regular, Medium, Bold, etc.)



Part 3: Color Styles, Text Styles, and Type Scale

Color Styles

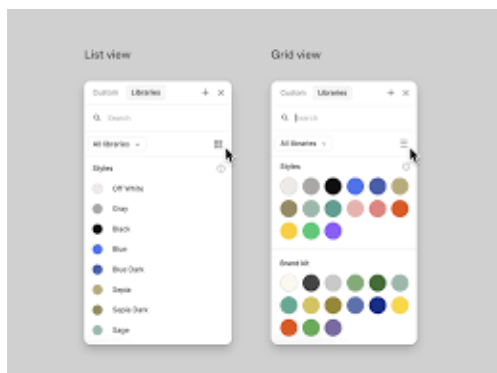
Color Styles allow designers to save and reuse colors across the entire project.

Steps to Create a Color Style:

- Select a shape.
- Choose the desired color.
- Click the four-dot icon in the Design Panel.
- Create a new style (e.g., “Primary Blue”, “Background Light”).

Benefits:

- Ensures design consistency.
- Allows quick global updates (change once, update everywhere).



Text Styles

Text Styles save complete typography settings for reuse.

Common Text Styles:

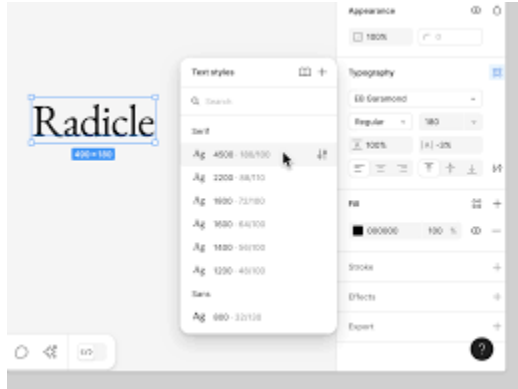
- Headings (H1, H2, H3)
- Body Text
- Caption / Label Text
- Type Scale Ratios

A type scale creates a harmonious hierarchy of text sizes. A commonly used ratio is 1.25 (Major Third).

Example Type Scale:

- H1: 32 px
- H2: 26 px
- H3: 20 px
- Body Text: 16 px
- Caption: 13 px

This creates visual balance and improves readability.



Part 4: Low-Fidelity Wireframes

What is a Wireframe?

A wireframe is a basic, structural blueprint of a user interface. It focuses on layout and functionality rather than visual design.

Purpose of Wireframes:

- Focus on overall structure and content placement.
- Plan user flow and navigation.

- Identify usability issues early.
- Facilitate feedback before investing in high-fidelity design.

Characteristics of Low-Fidelity Wireframes:

- Black and white (grayscale).
- Simple rectangles and basic shapes.
- Placeholder text (e.g., “Lorem Ipsum”).
- No colors, images, or detailed styling.

3-Page Wireframe Designs Created:

1. Login Page Wireframe

- Logo placeholder (rectangle)
- Email input field
- Password input field
- Login button

2. Home Page Wireframe

- Header bar with logo and navigation
- Navigation menu
- Content cards section
- Footer

3. Profile Page Wireframe

- Profile image placeholder
- Username and bio
- Information sections
- Edit button

HCI Concepts Applied:

- **Visual Hierarchy** - Achieved through proper type scale and layout.
- **Consistency** - Ensured using Color and Text Styles.
- **Usability** - Improved through structured wireframing.
- **Minimalism** - Focus on structure in low-fidelity designs.

Conclusion

In this lab, we explored advanced shape manipulation techniques, including fills, strokes, effects, and Boolean operations. We also studied color theory, typography principles, and learned how to create reusable Color Styles and Text Styles using type scales. Additionally, we designed low-

fidelity wireframes for three different pages. This lab significantly strengthened our foundational skills in UI design and user-centered design practices.

LAB NO#04

Mid-Fidelity Wireframes in Figma + Layout Grids (Desktop, Tablet, Mobile)

Part 1: Mid-Fidelity Wireframes

What is a Mid-Fidelity Wireframe?

A mid-fidelity wireframe is a more refined and detailed version of a low-fidelity wireframe. It strikes a balance between basic structure and realistic UI elements, but still avoids final visual design elements such as colors, images, and branding.

Key Characteristics:

- Proper alignment and consistent spacing
- Basic typography (headings, body text)
- Realistic sizes for buttons, input fields, and cards
- Neutral colors (various shades of gray)
- No final images, icons, or brand colors

Purpose of Mid-Fidelity Wireframes:

- Clearly show the structure and visual hierarchy of the interface
- Help test layout and overall usability
- Serve as a bridge between rough sketches (low-fidelity) and the final polished design (high-fidelity)

Part 2: Creating Mid-Fidelity Wireframes in Figma

Step-by-Step Process:

Step 1: Create Frames for Different Devices

- Press F to activate the Frame Tool.
- Create three separate frames with the following dimensions:
- Desktop: 1440 × 1024 px
- Tablet: 768 × 1024 px
- Mobile: 375 × 812 px

Step 2: Apply Neutral Colors

Use different shades of gray for:

- Buttons
- Cards
- Input fields

Use black or dark gray for all text elements only.

Step 3: Add Essential UI Components

- Header / Navigation Bar
- Content sections (e.g., cards)
- Buttons and Input Fields

Step 4: Ensure Consistent Spacing

- Properly align all elements
- Maintain equal margins and padding
- Follow grid columns for structured layout

Part 3: Layout Grids in Figma

What is a Layout Grid?

A Layout Grid is a system of columns, margins, and gutters that helps designers maintain alignment, consistency, and visual harmony across different screen sizes.

Recommended Layout Grid Settings:

Desktop Grid

- Grid Type: Columns
- Number of Columns: 12
- Margin: 80 px
- Gutter: 20 px
- Best Use: Websites and web dashboards

Tablet Grid

- Grid Type: Columns
- Number of Columns: 8
- Margin: 40 px
- Gutter: 16 px
- Best Use: Tablets and iPads

Mobile Grid

- Grid Type: Columns
- Number of Columns: 4
- Margin: 16 px
- Gutter: 8 px
- Best Use: Mobile applications

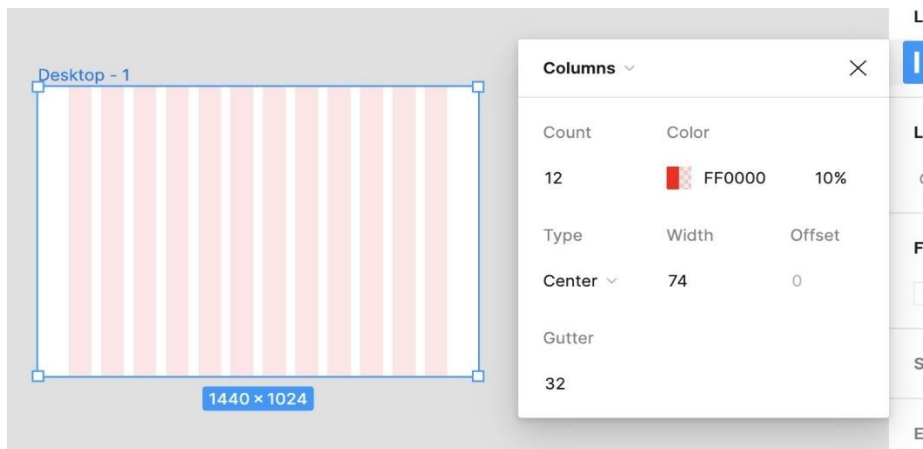
Part 4: Applying Grids to Mid-Fidelity Wireframes

Procedure:

- Select each frame and enable the Layout Grid from the frame settings.
- Align all UI elements according to the grid columns.
- Resize and position components while following the grid guidelines.
- Maintain visual consistency across Desktop, Tablet, and Mobile versions.

HCI Concepts Applied:

- **Consistency** - Achieved through the use of layout grids.
- **Responsiveness** - Designs adapt properly across multiple devices.
- **Visual Hierarchy** - Created using appropriate spacing and alignment.
- **Usability** - Clear and well-organized layouts improve user experience.



Conclusion

In this lab, we designed mid-fidelity wireframes for multiple devices and implemented layout grids to ensure proper alignment and structure. This systematic approach helps in creating responsive, consistent, and user-friendly interfaces while making the transition to high-fidelity designs much smoother and more efficient.

LAB NO#05

Dashboard Grid Layout Setup in Figma + Figma Plugins + Dashboard Design Assignment

Part 1: Dashboard Grid Layout Setup in Figma

What is a Dashboard?

A dashboard is a single-screen visual interface that provides an at-a-glance overview of key information, statistics, metrics, and interactive controls. Dashboards are widely used in web and mobile applications for data monitoring and quick decision-making.

Dashboard Grid Setup (Desktop)

Step-by-Step Procedure:

- Open Figma and create a new design file.
- Press F to select the Frame Tool.
- Create a Desktop frame with dimensions 1440 × 1024 px.
- Enable the Layout Grid in the frame settings.

Recommended Grid Settings:

- Grid Type: Columns
- Number of Columns: 12
- Margin: 80 px
- Gutter: 20 px

Purpose of Grid:

- Ensures proper alignment of elements
- Maintains consistent spacing throughout the design
- Creates a responsive and structured dashboard layout

Part 2: Figma Plugins

What are Figma Plugins?

Figma Plugins are small applications developed by the community that extend Figma's native functionality. They help designers work more efficiently by automating tasks, providing access to resources, and adding advanced features.

Plugins Used in This Lab:

Iconify

Provides access to thousands of high-quality icons. Used for sidebar navigation and dashboard icons.

Pixabay

Offers free stock images. Used for banners, backgrounds, or image placeholders.

Icons8 Background Remover

Quickly removes backgrounds from images. Especially useful for profile pictures and avatars.

Illustrations

Provides beautiful vector illustrations. Used for empty states, onboarding screens, or decorative banners.

Text Styles Creator

Automatically generates consistent typography styles. Helps maintain text hierarchy across the project.

Color Styles

Creates and manages reusable color styles for better visual consistency.

Beautiful Shadows

Generates professional shadow effects. Applied on cards, widgets, and modals for depth.

Fig3D

Allows insertion of 3D objects. Used for modern decorative elements.

Visual Mockups

Places the design into realistic device mockups. Helpful for presentations and client reviews.

Part 3: Dashboard Design Assignment

Objective:

Design a professional admin/dashboard interface using the 12-column grid and plugins.

Main Dashboard Components:

- Sidebar Navigation (with icons)
- Top Header (with search bar, notifications, and profile)

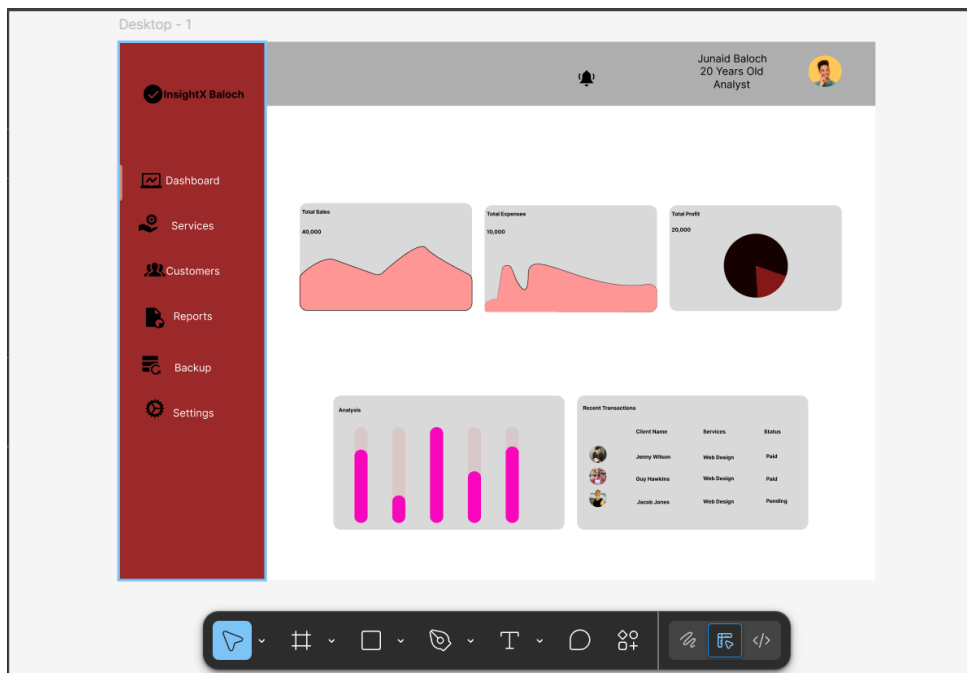
- Statistic Cards (Key Performance Indicators)
- Charts Placeholders (for graphs and analytics)
- Table or List Section (data display)

Design Guidelines Followed:

- Neutral color scheme with 1–2 accent colors
- Strict alignment using the layout grid
- Consistent margins, padding, and spacing
- Strategic use of icons to improve usability and visual appeal

HCI Concepts Applied:

- **Consistency** - Achieved through grid layout and reusable styles.
- **Visibility** - Clear information hierarchy and well-organized data presentation.
- **Efficiency** - Enhanced workflow using Figma plugins.
- **Usability** - Clean structure and intuitive navigation for better user experience.



Conclusion

In this lab, we successfully set up a structured grid layout for a dashboard, explored various useful Figma plugins, and designed a professional dashboard interface. This practical assignment enhanced our UI design skills, improved workflow efficiency, and gave us hands-on experience in creating real-world application interfaces.

LAB NO#06

Auto Layout in Figma + Buttons and Cards + Responsive Design

Part 1: Auto Layout Overview

What is Auto Layout?

Auto Layout is one of the most powerful and time-saving features in Figma. It automatically arranges and organizes UI elements such as buttons, cards, lists, and navigation bars according to defined spacing rules and alignment settings. This feature enables designers to create responsive, flexible, and easily maintainable designs.

Key Auto Layout Properties:

- **Spacing between items:** Controls the distance/gap between child elements inside a frame or component.
- **Padding:** Defines the inner space between the container's edges and the elements inside it (top, bottom, left, and right).
- **Direction / Flow:** Determines whether elements are arranged horizontally or vertically.
- **Wrapping:** Allows elements to automatically move to the next line when there is insufficient space in the current row.
- **Fixed Width:** Keeps the width of the frame constant regardless of the content inside.
- **Hug Contents:** The frame automatically resizes itself to perfectly fit its content.
- **Fill Container:** Child elements expand to occupy the full available space inside the parent container.

Part 2: Creating Buttons with Auto Layout

Step-by-Step Process:

- Draw a rectangle using the Rectangle Tool and place a text layer on top of it.
- Select both the rectangle and the text layer, right-click, and select Add Auto Layout.
- In the Auto Layout panel, adjust the Padding values (Top, Bottom, Left, Right) to create balanced spacing.
- Set the spacing between the text and the edges as required.
- Change the frame's width setting from "Fixed Width" to Hug Contents.

Result:

The button automatically adjusts its width when the text content is changed.

Consistent internal padding and professional alignment are maintained at all times.

Part 3: Creating Cards with Auto Layout

Step-by-Step Process:

- Create a new frame that will act as the main card container.
- Add all necessary elements inside the card (image or image placeholder, title, description, and button).
- Select the entire card frame and apply Auto Layout.
- Carefully adjust the internal padding and spacing between all elements.
- Set both width and height properties to Hug Contents.

Result:

- The card dynamically resizes according to its content.
- All elements inside the card stay properly spaced and aligned.

Part 4: Responsive Cards with Auto Layout

Step-by-Step Process:

- Place multiple card components inside a parent frame.
- Apply Auto Layout to the parent frame.
- Set the direction to Horizontal.
- Enable the Wrapping option.
- Adjust the spacing between individual cards.

Result:

- When the screen width decreases, cards automatically wrap to the next row.
- The entire layout remains clean, well-aligned, and visually balanced across different screen sizes.

Part 5: Auto Layout Practice Exercises

- Create 3 buttons with different text lengths.
- Design 4 cards that include images, titles, descriptions, and action buttons.
- Place all cards inside a parent frame in a horizontal row with wrapping enabled.
- Resize the parent frame to test the responsiveness of the layout.
- Fine-tune padding, spacing, and alignment to achieve a clean and professional look.

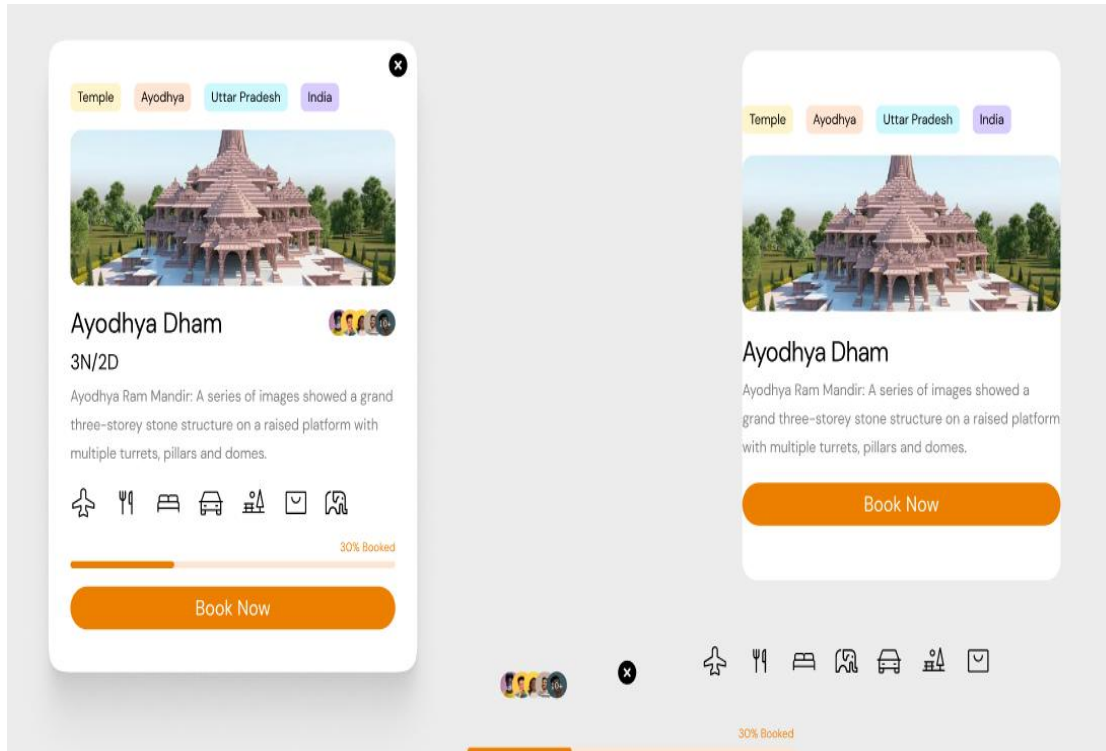
HCI Concepts Applied:

Consistency - Buttons and cards maintain uniform spacing and alignment.

Responsiveness - The layout automatically adapts to different screen sizes.

Efficiency - Auto Layout allows rapid layout changes without manual repositioning of elements.

Usability - Flexible and resizable components provide a better overall user experience.



Conclusion

In this lab, we thoroughly explored the Auto Layout feature in Figma. We learned how to create responsive buttons and cards that automatically adjust according to content and screen size. Auto Layout greatly simplifies the design workflow, ensures visual consistency, and is considered an essential skill for modern professional UI/UX design.

LAB NO#07

Components and Variants in Figma

Part 1: Components in Figma

What is a Component?

A Component in Figma is a reusable design element that can be used multiple times across a project. Once a component is created, any changes made to the main component are automatically reflected in all its instances. Components are essential for maintaining design consistency and improving workflow efficiency.

Key Benefits of Using Components:

- Ensure visual and functional consistency across the entire design.
- Allow creation of multiple instances that inherit properties from the main component.
- Make large-scale design updates quick and easy.

Key Component Operations:

Create Component - Converts a selected object or group into a reusable component (Shortcut: Ctrl + Alt + K).

Create Instance - Creates a copy of the main component that remains linked to it.

Go to Main Component - Quickly navigates from any instance back to the original main component.

Reset Size - Restores the component frame to its original default size.

Reset Instance - Reverts any changes made to an instance back to the main component's properties.

Push Changes - Applies modifications made in an instance back to the main component.

Detach Instance - Breaks the link between the instance and the main component, converting it into a regular editable object.

Part 2: Variants in Figma

What is a Variant?

Variants are different versions or states of the same component, grouped together in a single Component Set. They allow designers to manage multiple states of an element efficiently.

Example: A button component can have variants such as:

- Default
- Hover
- Pressed
- Disabled
- Active

Steps to Create Variants:

- Create the main component (e.g., a Button).
- Duplicate the component and modify it to represent a different state (e.g., Hover state by changing color or shadow).
- Select all related components and combine them as Variants.
- Clearly name each variant (e.g., "Default", "Hover", "Pressed") in the Variants panel.

Benefits of Variants:

- Easy switching between different states during prototyping.
- Reduces unnecessary duplication of similar components.
- Simplifies design management and improves organization.

Part 3: Multiple Components and Component Sets

Multiple Components

Common UI elements such as Sidebars, Cards, Buttons, and Input Fields are converted into individual components. This allows designers to make uniform changes across all instances throughout the project.

Component Set

A Component Set is a collection of related variants grouped under one component. For example, a single Card component can contain multiple variants like Default, Hover, and Selected states.

Part 4: Card Components Assignment

Practical Task:

Design a Card containing the following elements:

- Image or image placeholder
- Title
- Description text
- Action Button

Convert the entire card frame into a Component.

Create multiple instances of this card component for use in a dashboard.

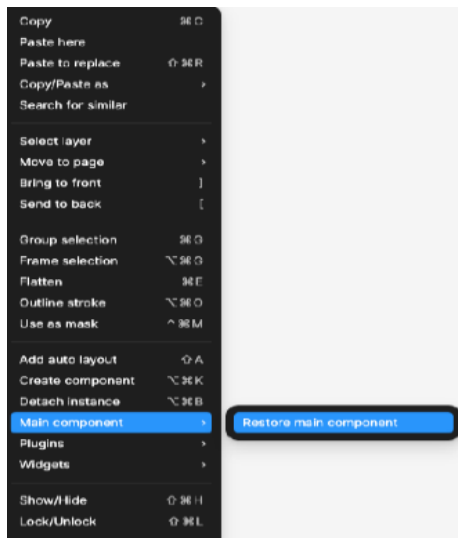
Update content in one instance and use Push Changes to update the main component.

Create different Variants for the card (e.g., Default and Hover states).

Use Detach Instance when unique modifications are needed that should not affect the main component.

HCI Concepts Applied:

- **Consistency** - Uniform appearance of components across the entire user interface.
- **Efficiency** - Quick global updates using the main component.
- **Flexibility** - Variants allow multiple interactive states.
- **Reusability** - Saves significant time and effort in the design workflow.



Conclusion

In this lab, we learned how to create and manage Components, Instances, and Variants in Figma. We successfully designed reusable card components and understood how to maintain consistency while working efficiently. Components and Variants are fundamental for building scalable, consistent, and professional design systems in modern UI/UX projects.

LAB NO#08

Component Sets (Variants) + Prototyping in Figma

Part 1: Component Set (Different Variants)

What is a Component Set?

A **component set** is a collection of related components, each representing a **different variant** or state.

- Example: Buttons with **Default, Hover, Pressed, Disabled** states
- Helps maintain **consistency** and **simplifies prototyping**

Steps to Create a Component Set

- **Create a main component** (e.g., Button or Card)
- **Duplicate** the component and modify it to represent another state (Hover, Disabled, Selected)
- Select all related components → **Combine as Variants**
- Use the **Variants panel** to name each variant (e.g., Default, Hover)

Benefits:

- Easy switching between states in **prototyping**
- Reduces repetition and increases efficiency
- Ensures **consistent design** across the project

Part 2: Prototyping in Figma

What is Prototyping?

Prototyping allows designers to create **interactive flows** between screens to simulate the final user experience.

Key Features:

- Link screens/components
- Add **interactive triggers** (Click, Hover, Drag)
- Apply **transitions / animations**
- Test **user flow** before development

Here are some steps for Prototyping

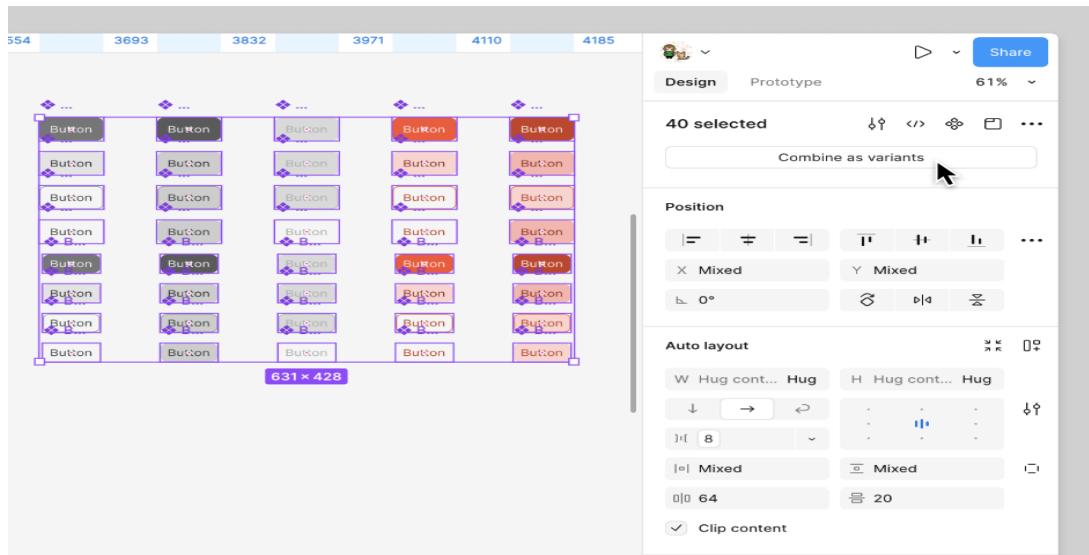
- Select a **component** or **frame**
- Click **Prototype tab** in right panel
- **Drag connection** to target frame
- Set **interaction type**:
 - On Click
 - While Hovering
 - After Delay
- Choose **Animation / Transition**:
 - Instant
 - Dissolve
 - Smart Animate
- Preview prototype using **Play button** (top-right corner)

Example: Button Component with Variants

- Button component set has **Default, Hover, Pressed**
- Prototype interaction:
 - On Hover → Switch to Hover variant
 - On Click → Navigate to another frame/page

HCI Concepts Applied

- **Feedback**: Hover/Pressed states show interactivity
- **Consistency**: Same component used across multiple frames
- **Efficiency**: Quick prototyping with reusable variants
- **Usability**: Test user flows before final design



Conclusion

In this lab, we created **component sets with multiple variants** and built **interactive prototypes** in Figma. This helps designers test **user interactions** efficiently and maintain a **consistent, reusable design system**.

LAB NO#09

Drawing Primitives of shapes

Tools Required:

- Laptop
- Internet Access
- P5.js (online website used in code-based design)

What is P5.js:

p5.js is a free, open-source **JavaScript library** designed for creative coding. It provides a "software sketchbook" environment that makes it easy for artists, designers, and beginners to create interactive graphics, animations, and art for the web.

Drawing Primitives

Objective:

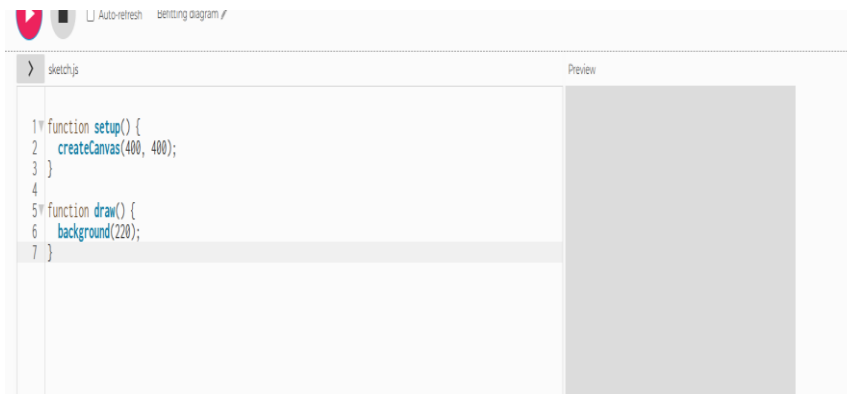
Learn how to draw the basic building blocks of computer graphics - points, lines, rectangles, circles, ellipses, and triangles - using p5.js.

Step 1 – Creating a Canvas

Explanation:

- The canvas is your digital drawing paper.
- The coordinate system starts at the top-left corner (0,0).
- Increasing x → right, increasing y → down.
- background(220) sets a light gray background (values from 0–255).

When you run this, you should see a blank gray box.



Step 2 – Drawing a Point



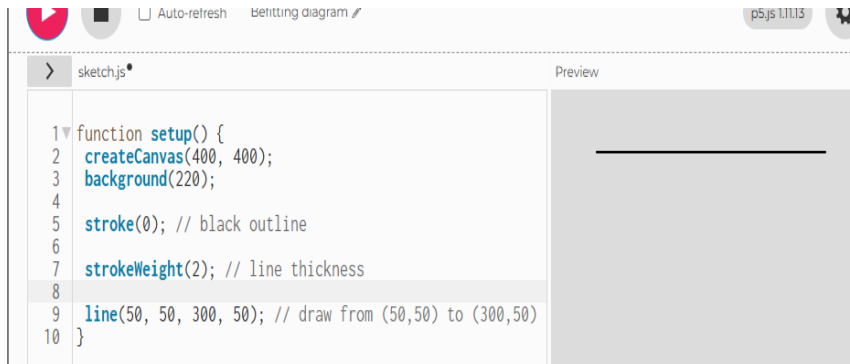
Explanation:

- A point is the smallest visible element — one pixel.
 - point(x, y) activates a single pixel on the screen.
 - strokeWeight() controls how thick the point looks.
- Try drawing multiple points in a pattern to make simple dotted lines or designs.

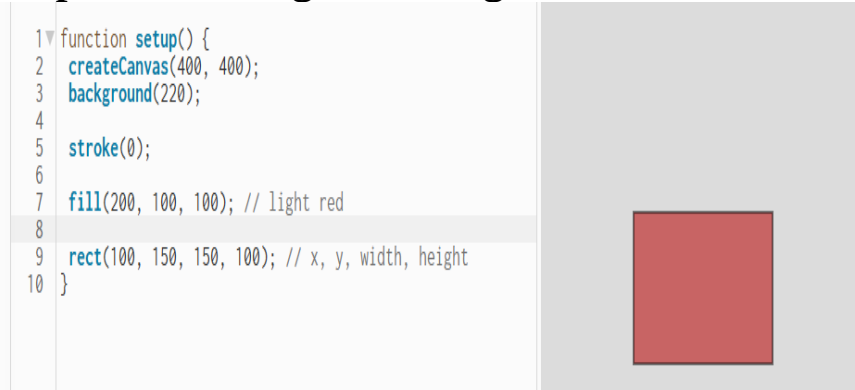
Step 3 – Drawing a Line

Explanation:

- A line connects two points (x1, y1) and (x2, y2).
 - The computer draws many points between those coordinates.
 - Later (DDA & Bresenham), students will see how this line is plotted pixel by pixel.
- Try different coordinates to make diagonal or vertical lines.



Step 4 – Drawing a Rectangle

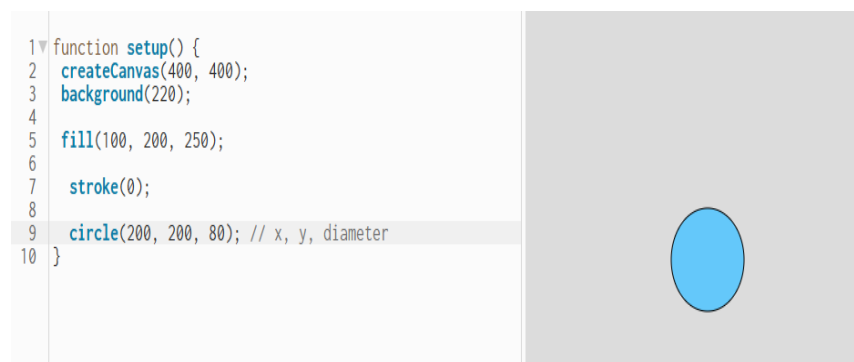


Explanation:

- The first two numbers (100,150) are the starting point (top-left corner).
- Next two numbers are width and height.
- fill() adds color inside, while stroke () outlines the shape.

Change the position or size and observe how it moves.

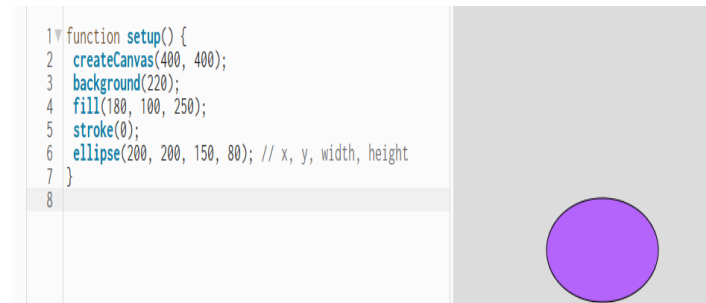
Step 5 – Drawing a Circle



Explanation:

- circle (x, y, d) draws a circle with center (x, y) and diameter d.
- Perfect for creating wheels, sun, eyes, or buttons in UIs.
- Changing d resizes it evenly in all directions.

Step 6 – Drawing an Ellipse

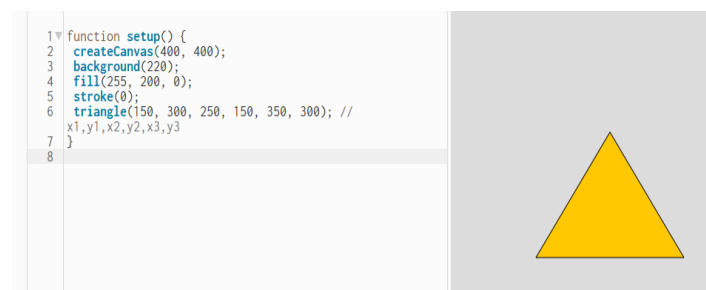


Explanation:

- ellipse(x, y, w, h) works like circle() but allows different width and height.
- When width = height → it becomes a circle.
- Used to draw ovals, faces, eyes, or stretched shapes.

Try Making two small ellipses for eyes and a circle for a face

Step 7 – Drawing a Triangle



Explanation:

- triangle() connects three points.
- The points are the vertices (corners).
- Great for roofs, arrows, and simple polygon shapes.

Step 8 – Combine Shapes into a Simple Scene



Explanation:

We're combining multiple primitives — rectangles, triangles, and ellipses — to make a small scene. In real computer graphics, complex models are built this same way — starting from simple shapes.

LAB NO#10 + 11

Line Algorithms (DDA & Bresenham)

1. Digital Differential Analyzer (DDA) Line Drawing Algorithm Objective.

To understand how a computer draws a line pixel by pixel using the DDA algorithm, instead of just using the built-in line () function.

Introduction

In computer graphics, every line on the screen is made up of tiny square dots called pixels. When we write:

`line (50, 100, 300, 250);`

in p5.js, the computer secretly uses an algorithm (like DDA or Bresenham) to decide which pixels should be turned on to make the line appear.

So, we study DDA not because we can't use line(), but because:

- We want to learn how computers actually draw lines internally.
- It helps us understand rasterization — converting mathematical lines into pixel lines.
- It builds a foundation for advanced topics like OpenGL, 3D graphics, and animation.

Concept Behind DDA

The Digital Differential Analyzer (DDA) algorithm:

- Starts from the first point (x1, y1).
- Calculates small equal steps between start and end points.
- Moves in small increments in both x and y directions.
- Plots one pixel at each step until it reaches (x2, y2).

In short:

DDA divides the line into small equal steps and draws one pixel at a time.

Code (p5.js Version)

```
sketch.js Preview
1 function setup() {
2   createCanvas(400, 400); // create drawing area
3   background(255); // white background
4   // draw a line from (50,100) to (300,250)
5   DDA(50, 100, 300, 250);
6 }
7 function DDA(x1, y1, x2, y2) {
8   // Step 1: Calculate the difference in x and y
9   let dx = x2 - x1; // horizontal distance
10  let dy = y2 - y1; // vertical distance
11  // Step 2: Find how many steps (pixels) we need
12  let steps;
13  if (abs(dx) > abs(dy)) {
14    steps = abs(dx); // more horizontal movement
15  } else {
16    steps = abs(dy); // more vertical movement
17  }
18  // Step 3: Calculate how much to move each time
19  let xIncrement = dx / steps;
20  let yIncrement = dy / steps;
21
22  // Step 4: Start from the first point
23  let x = x1;
24  let y = y1;
25
26  // Step 5: Plot one pixel for each step
27  for (let i = 0; i <= steps; i++) {
28    point(round(x), round(y)); // draw nearest whole pixel
29    x = x + xIncrement; // move in x
30    y = y + yIncrement; // move in y
31  }
32 }
33
```

In Simple Words

1. We start at the first point.
2. We figure out how far we need to go.
3. We break that distance into tiny steps.
4. In each step, we move a little in x and y.
5. We draw one pixel per step — that becomes a line!

Key Takeaways

- DDA converts mathematical lines into pixel lines.
- It's based on increments and steps.
- The round () function ensures only whole pixels are drawn.
- It's simple, visual, and foundational for all raster graphics.

2. Bresenham Line Drawing Algorithm

Introduction

When drawing a line between two points on a computer screen, the ideal mathematical line passes between pixels.

But a computer screen (raster display) can only draw whole pixels. So, a line drawing algorithm must decide:

“At each step, which pixel is closest to the ideal line?”

DDA uses floating-point arithmetic and rounding. Bresenham improves this by using only integers, which makes it more efficient and more accurate.

Concept Behind Bresenham

The algorithm always moves one step in x, and then decides whether y should:

- stay the same (horizontal movement), or
- change by ± 1 (diagonal movement)

Explanation of Steps

Step	Description	Example (50,100 → 300,250)
1	Find dx and dy — how far to move in x and y.	dx = 250, dy = 150
2	Choose the longer distance as total steps.	steps = 250
3	Divide distance by steps to find small movements.	xInc = 1, yInc = 0.6
4	Start from first point.	x = 50, y = 100
5	Draw one pixel at each step and move slightly.	(51,101), (52,101), (53,102)... until (300,250)

The “decision parameter” p tells us which move is closer to the actual line.

At every step:

- If $p < 0 \rightarrow$ the line is still below midpoint $\rightarrow$ move straight in x
- If $p \geq 0 \rightarrow$ the line has crossed midpoint $\rightarrow$ move diagonally

The decision parameter is updated after every pixel using only addition/subtraction.

Understanding the Decision Parameter (p)

This is the most important concept. Between two possible next pixels:

- $(x+1, y) \rightarrow$ horizontal
- $(x+1, y+1)$ or $(x+1, y-1) \rightarrow$ diagonal

We imagine a midpoint between these two choices:

- midpoint = $(x + 1, y + 0.5)$

We test whether the ideal line is above or below this midpoint. To avoid decimals (0.5), we multiply everything by 2. That removes fractions and keeps all calculations integer-only.

That is why Bresenham initializes: $p = 2*dy - dx$ Then:

- If $p < 0 \rightarrow$ horizontal
- If $p \geq 0 \rightarrow$ diagonal

This allows perfect pixel selection without any floating-point rounding.

Comparison: DDA vs Bresenham

Feature	DDA Algorithm	Bresenham Algorithm
Arithmetic	Floating point	Integer only
Speed	Slower	Faster
Accuracy	Uses rounding	Exact selection
Concept	Adds fractions	Uses decision parameter
Slope handling	Works for all	Simple version works only if
Practical use	Learning	Real hardware rasterization

Bresenham’s Algorithm (Simple Version)

Only works for gentle lines ($|\text{slope}| \leq 1$)

Formula for Slope

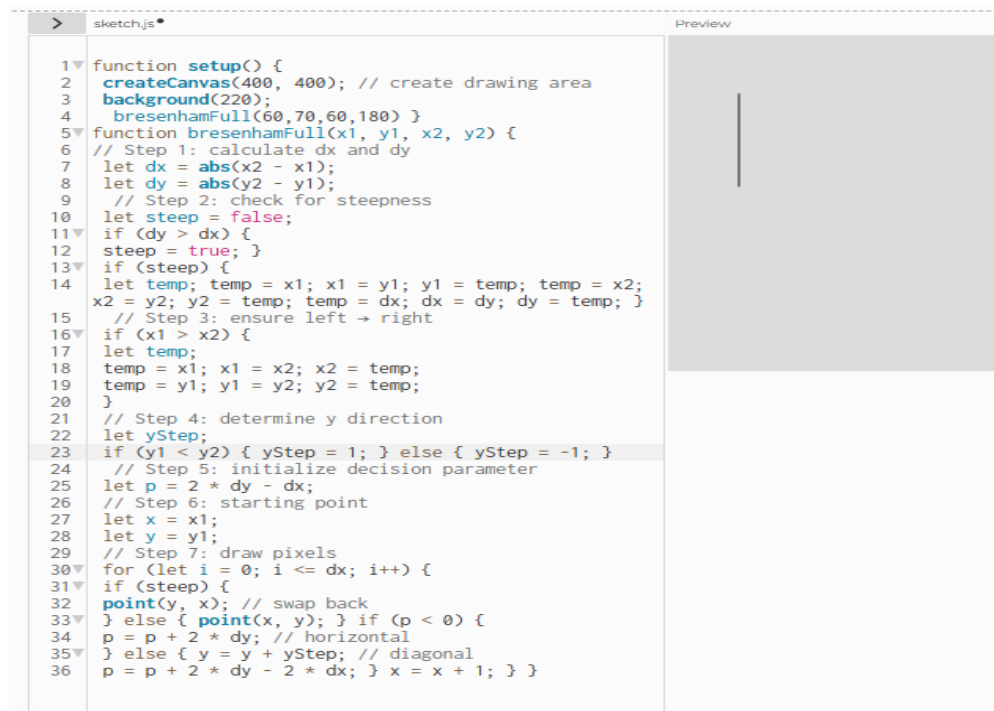
$$\bullet \quad m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{dy}{dx}$$

Case	Slope value	What it means	DDA moves by
Case 1: (	m	< 1)	Line is more horizontal
Case 2: (	m	> 1)	Line is more vertical
Case 3: (	m	$= 1$)	Perfect diagonal line

Steps:

1. Compute dx and dy
2. Ensure x increases from left to right (swap points if needed)
3. Determine y direction (yStep = +1 or -1)
4. Initialize the decision parameter p
5. Start from first point (x1, y1)
6. Repeat for dx steps
 - > Plot pixel
 - > If $p < 0 \rightarrow$ move horizontally
 - > Else $\rightarrow$ move diagonally
 - > Update p
 - > Increase x

Code (simple version)



```
> sketch.js
1 function setup() {
2   createCanvas(400, 400); // create drawing area
3   background(220);
4   bresenhamFull(60,70,60,180) }
5 function bresenhamFull(x1, y1, x2, y2) {
6   // Step 1: calculate dx and dy
7   let dx = abs(x2 - x1);
8   let dy = abs(y2 - y1);
9   // Step 2: check for steepness
10  let steep = false;
11  if (dy > dx) {
12    steep = true; }
13  if (steep) {
14    let temp; temp = x1; x1 = y1; y1 = temp; temp = x2;
    x2 = y2; y2 = temp; temp = dx; dx = dy; dy = temp; }
15  // Step 3: ensure left to right
16  if (x1 > x2) {
17    let temp;
18    temp = x1; x1 = x2; x2 = temp;
19    temp = y1; y1 = y2; y2 = temp;
20  }
21  // Step 4: determine y direction
22  let yStep;
23  if (y1 < y2) { yStep = 1; } else { yStep = -1; }
24  // Step 5: initialize decision parameter
25  let p = 2 * dy - dx;
26  // Step 6: starting point
27  let x = x1;
28  let y = y1;
29  // Step 7: draw pixels
30  for (let i = 0; i <= dx; i++) {
31    if (steep) {
32      point(y, x); // swap back
33    } else { point(x, y); } if (p < 0) {
34      p = p + 2 * dy; // horizontal
35    } else { y = y + yStep; // diagonal
36      p = p + 2 * dy - 2 * dx; } x = x + 1; } }
```

Limitations of the Simple Version

The simple version assumes:

- we always move 1 pixel in x
- x increases more than y ($dx \geq dy$)
- slope between -1 and 1.

If slope > 1 or slope < -1 , the line will distort. To fix this, we need the Full Bresenham Algorithm.

Bresenham's Algorithm (Full Version – Handles All Slopes)

Idea

If the line is “steep” ($dy > dx$), we swap x and y so that we draw along the dominant axis. This is called the “steep line fix”.

Steps

1. Compute dx , dy
2. Check if line is steep
 - o If steep $\rightarrow$ swap x and y for both points
3. Ensure left $\rightarrow$ right (swap points if $x_1 > x_2$)
4. Determine y direction ($yStep$)
5. Initialize decision parameter p
6. Start from (x_1, y_1)
7. For dx steps
 - o If steep $\rightarrow$ plot (y, x)
 - o Else $\rightarrow$ plot (x, y)
 - o Use p to decide horizontal vs diagonal
 - o Update p
 - o Increase x

Code (Full Version)

```
1 function setup() {
2   createCanvas(400, 400); // create drawing area
3   background(220);
4   bresenhamSimple(60,70,60,180) }
5 function bresenhamSimple(x1, y1, x2, y2) {
6   // Step 1: calculate differences
7   let dx = abs(x2 - x1);
8   let dy = abs(y2 - y1);
9   // Step 2: ensure left  $\rightarrow$  right
10  if (x1 > x2) {
11    let temp;
12    temp = x1;
13    x1 = x2;
14    x2 = temp;
15    temp = y1;
16    y1 = y2;
17    y2 = temp;
18  }
19  // Step 3: determine y direction
20  let yStep;
21  if (y1 < y2) {
22    yStep = 1;
23  } else {
24    yStep = -1;
25  }
26  // Step 4: initialize decision parameter
27  let p = 2 * dy - dx;
28  // Step 5: start point
29  let x = x1;
30  let y = y1;
31  // Step 6: draw pixels
32  for (let i = 0; i <= dx; i++) {
33    point(x, y);
34    if (p < 0) {
35      p = p + 2 * dy; // horizontal
36    } else {
37      y = y + yStep; // diagonal
38      p = p + 2 * dy - 2 * dx;
39    }
40    x = x + 1; }
```

Difference Between Simple and Full Version

Feature	Simple Version	Full Version
Handles slope		m
Steep lines	Not handled	Swapping x-y fixes it
Swapping	Only swap for left $\rightarrow$ right	Swap for steepness + left $\rightarrow$ right
Use case	Basic learning	Real implementation

Key Takeaways

- Bresenham draws lines pixel-by-pixel using only integers.
- Decision parameter p decides whether the next step is horizontal or diagonal.
- Simple version works only for gentle slopes.
- Full version swaps coordinate to handle all slopes.
- Works perfectly on raster screens (pixel grids).
- Faster and more accurate than DDA.

Midpoint Circle Algorithm

Introduction

A circle can be drawn using the equation:

$$x^2 + y^2 = r^2$$

But using this directly is slow because it requires square roots and floating-point math. Computer graphics use faster algorithms that draw the circle pixel by pixel. The Midpoint Circle Algorithm is one such method.

It uses:

- Only integer arithmetic
- No square roots
- Symmetry of the circle

This makes it fast and perfect for raster displays.

Basic Idea

A circle has 8-way symmetry. If you calculate one point in one octant, you automatically get 7 more by reflection:

So, we only compute points from:

$$x = 0 \text{ to } 45^\circ$$

Then we reflect on them around the center.

Why We Use a Decision Parameter

When moving from one x to the next $x+1$, there are two possible pixels:

1. Right pixel $\rightarrow (x+1, y)$

2. Diagonal pixel $\rightarrow (x+1, y-1)$

We need to choose the pixel that is closest to the true circle. So, we evaluate the midpoint between these two choices. If the midpoint is:

- Inside circle $\rightarrow$ choose Right
- Outside circle $\rightarrow$ choose Diagonal

We store this test in a variable called p (decision parameter).

Initial Conditions

At the top of the circle:

$$x = 0 \quad y = r$$

Initial decision parameter:

$$p = 1 - r$$

Updating the Decision Parameter

If midpoint inside circle ($p < 0$)

Choose Right pixel

$$p = p + 2 \cdot x + 1$$

If midpoint outside circle ($p \geq 0$)

Choose Diagonal pixel

$$y = y - 1$$

$$p = p + 2 \cdot (x - y) + 1$$

Steps of the Midpoint Circle Algorithm

Step 1: Initialize

$$x = 0$$

$$y = r$$

$$p = 1 - r$$

Step 2: Plot the 8 symmetric points

Use helper function drawCirclePoints(xc, yc, x, y).

Step 3: Repeat while $x < y$

This covers the first 45° octant.

Step 4: Increase x at each step

$$x = x + 1$$

Step 5: Use decision parameter

Choose Right or Diagonal pixel.

Step 6: Plot symmetric points again

Repeat until complete.

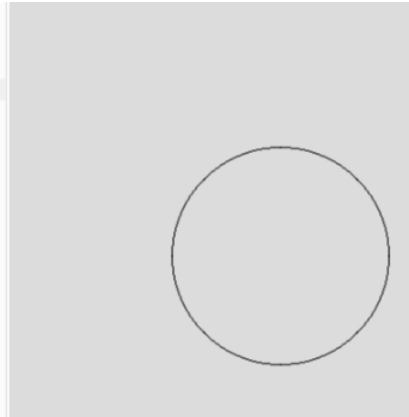
8-Way Symmetry Function (Important)

```
function drawCirclePoints(xc, yc, x, y) {  
    point(x+xc,y+yc); point(-x+xc,y+yc);  
    point(-x+xc,-y+yc);  
    point(x+xc,-y+yc);  
    point(y+xc,x+yc);  
    point(-y+xc,x+yc);  
    point(-y+xc,-x+yc);  
    point(y+xc,-x+yc); }
```

This draws all 8 symmetric points around the center.

Complete Midpoint Circle Code (p5.js)

```
1 function setup() {  
2   createCanvas(400, 400); // create drawing area  
3   background(220);  
4   midpointCircle(250, 250, 100); //Example Cell  
5 }  
6 function midpointCircle(xc, yc, r) {  
7   let x = 0;  
8   let y = r;  
9   let p = 1 - r;  
10  drawCirclePoints(xc, yc, x, y);  
11  while (x < y) {  
12    x = x + 1;  
13    if (p < 0) {  
14      p = p + 2 * x + 1;  
15    } else {  
16      y = y - 1;  
17      p = p + 2 * (x - y) + 1;  
18    }  
19    drawCirclePoints(xc, yc, x, y);  
20  }  
21 }  
22 function drawCirclePoints(xc, yc, x, y) {  
23   point(x+xc,y+yc);  
24   point(-x+xc,y+yc);  
25   point(-x+xc,-y+yc);  
26   point(x+xc,-y+yc);  
27   point(y+xc,x+yc);  
28   point(-y+xc,x+yc);  
29   point(-y+xc,-x+yc);  
30   point(y+xc,-x+yc);  
31 }
```



Matching p5.js circle () Important:

circle (x, y, d) → d = diameter

midpointCircle(x, y, r) → r = radius

So:

circle (250, 250, 200); // radius 100

midpointCircle(250, 250, 100);

Comparison: Bresenham Line vs Midpoint Circle

Feature	Bresenham Line	Midpoint Circle
Purpose	Draw line	Draw circle
Symmetry	None	8-way symmetry
Pixels move	Right or diagonal	Right or diagonal
Decision parameter	Yes	Yes
Complexity	Low	Medium
Initialization	From (x1, y1)	From (0, r)
Uses integer math	Yes	Yes

Key Takeaways

- Circles can be drawn without trigonometry or square roots.
- Only integer addition/subtraction is used.

- 8-way symmetry reduces work by 87.5%.
- Decision parameter decides if we move horizontally or diagonally.

LAB NO#13

2D Transformation

2D Transformations allow us to change the position, size, and orientation of shapes. Every transformation modifies the coordinate system, not the shape directly.

Main types:

1. Translation → moves the shape
2. Scaling → increases/decreases size
3. Rotation → rotates around a pivot point

Mathematical Formulas

A. Translation

Moves object without changing shape.

$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

B. Scaling

Enlarges or shrinks shape.

$$\begin{aligned}x' &= x * sx \\ y' &= y * sy\end{aligned}$$

C. Rotation (about origin)

Rotates point (x,y) by angle θ :

$$\begin{aligned}x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos\end{aligned}$$

p5.js Functions

p5.js provides built-in transformation functions:

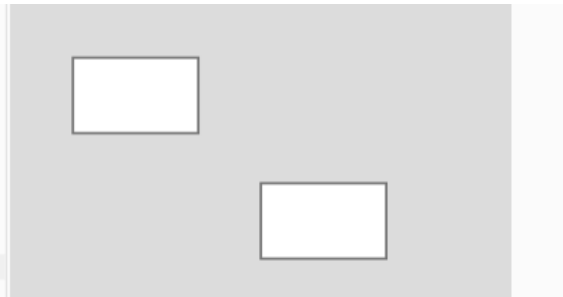
Purpose	Function
Move object	translate(x,y)
Scale object	scale(s)
Rotate object	rotate(angle)

Use push() and pop() to protect transformations:
 push(); // save current settings // transformations inside
 pop(); // restore settings.

Example Codes (p5.js)

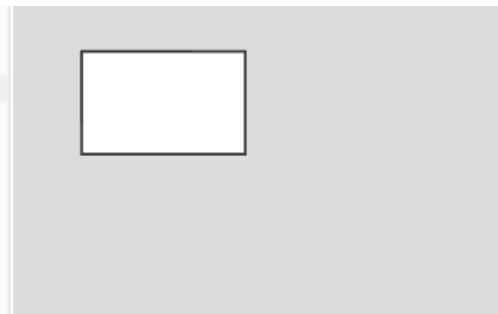
A. Translation

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4   rect(50, 50, 100, 60);  
5   push();  
6   translate(150, 100);  
7   rect(50, 50, 100, 60);  
8   pop();  
9 }  
10
```



B. Scaling

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4   rect(50, 50, 80, 50);  
5   push();  
6   translate(50, 50);  
7   scale(1.5); // only width/height scales  
8   rect(0, 0, 80, 50);  
9   pop();  
10 }  
11
```



Scaling — Very Important Concept

When you write:

`scale(1.5);`

This multiplies EVERYTHING drawn after it by 1.5:

- x-position
- y-position
- width
- height
- stroke weight
- translations
- rotations

Scaling affects everything, because it modifies the coordinate system.

Scaling Only Width & Height (Not Position)

If you write:

`scale(1.5);`

`rect(50, 50, 80, 50);`

This happens:

- x becomes $50 \times 1.5 = 75$

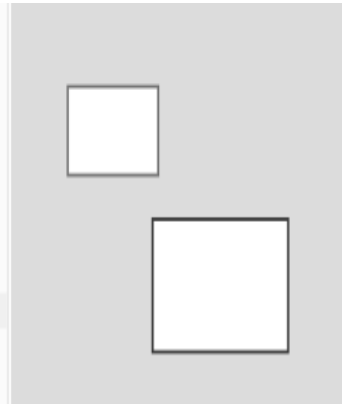
- y becomes $50 \times 1.5 = 75$
- width becomes 120
- height becomes 75

This usually gives unexpected results.

```

1 function setup() {
2   createCanvas(400, 400);
3   background(220);
4   rect(50, 50, 80, 50);
5   push();
6   translate(50, 50);
7   scale(1.5); // only width/height scales
8   rect(50, 50, 80, 50);
9   pop();
10 }

```



To avoid scaling the position:

push ();

translate (50, 50); // position NOT scaled

scale (1.5); // only size scales

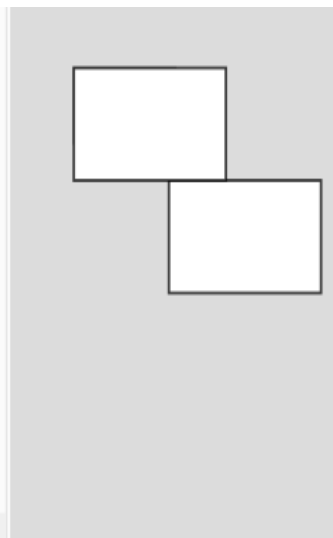
rect(0, 0, 80, 50); // shape starts at origin

pop ();

```

1 function setup() {
2   createCanvas(400, 400);
3   background(220);
4   rect(50, 50, 80, 50);
5   push();
6   translate(50, 50);
7   scale(1.5); // only width/height scales
8   rect(0, 0, 80, 50);
9   pop();
10
11   push ();
12   translate (50, 50); // position NOT scaled
13   scale (1.5); // only size scales
14   rect(0, 0, 80, 50); // shape starts at origin
15   pop ();
16
17 }

```



Scaling With Two Values

scale (sx, sy);

Meaning:

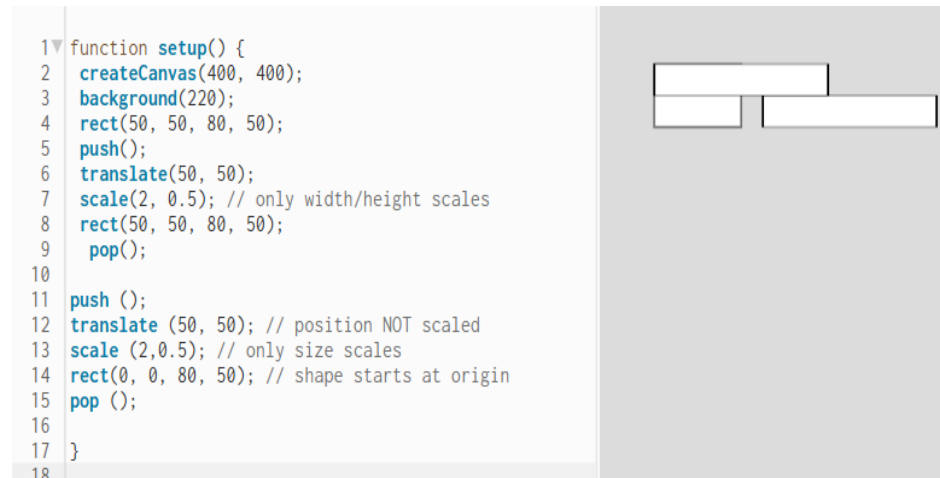
- All x values multiply by sx
- All y values multiply by sy

So:

- width scales with sx
- height scales with sy

Example:

scale (2, 0.5); // Wider, but shorter



Do All Graphics Systems Work This Way?

YES. This is universal in graphics.

Scaling ALWAYS affects:

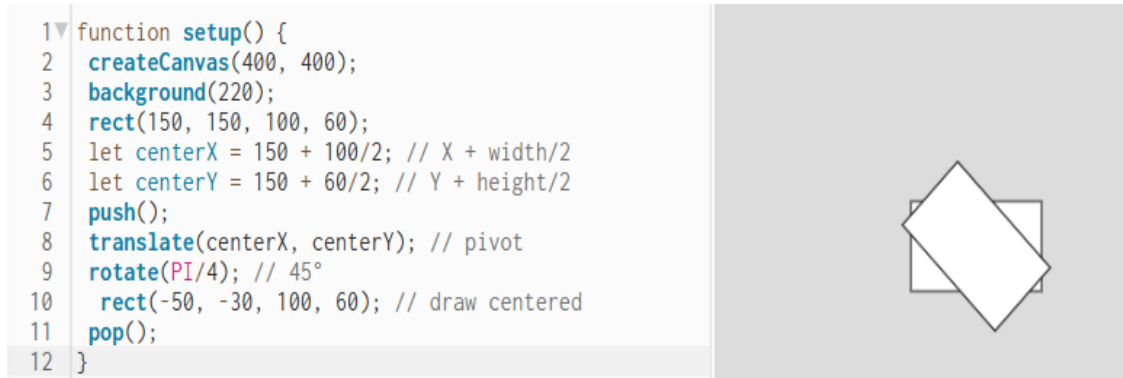
- x, y
- width, height
- vertices • transformations
- world coordinates

This is true in:

- p5.js
- Canvas API
- WebGL
- OpenGL
- DirectX
- Unity
- Blender
- SVG

Any matrix-based CG system.

C. Rotation



Rotation Reference Table (Radians)

Description	Radians	p5.js Code
No rotation	0	rotate(0)
45°	$\pi/4$	rotate(PI/4)
90°	$\pi/2$	rotate(PI/2)
180°	π	rotate(PI)
270°	$3\pi/2$	rotate(3*PI/2)
360°	2π	rotate(TWO_PI)

You can also write any angle like:

rotate (1.2); // 1.2 radians

rotate (0.8); // 0.8 radians

rotate (PI/6); // 30 degrees

rotate (PI/3); // 60 degrees

Rotation Using Degrees (Alternative Method)

In p5.js, rotation can also be done using degrees instead of radians. By default, p5.js uses radians (PI values), but we can change this using:

angleMode(DEGREES);

After setting this, we can directly use angles like:

- 45 instead of PI/4
- 90 instead of PI/2
- 180 instead of PI

Rotation Reference Table (Degrees)

Description	Degrees	p5.js Code
No rotation	0°	rotate(0)
45°	45	rotate(45)
90°	90	rotate(90)
180°	180	rotate(180)
270°	270	rotate(270)
360°	360	rotate(360)

Important Notes

angleMode(DEGREES) should be set in setup() (runs once).

Do NOT mix radians and degrees in the same program.

Rotation logic remains exactly the same:

translate to center

rotate

draw using negative half-width & half-height

Example (Using Degrees)

```
push();
```

```
translate(200, 180); // move origin to center
```

```
rotate(45); // 45 degrees
```

```
rect(-50, -30, 100, 60); // centered drawing
```

```
pop();
```

Why Rotate Needs Translate() First

Rotation ALWAYS happens around the **current origin (0,0)**.

To rotate shape around its own center, we must:

1. Move origin to shape center → translate(centerX, centerY)
2. Rotate
3. Draw shape with negative half-width/half-height

Example:

Rectangle at (150,150), width=100, height=60

Compute center:

$$\text{centerX} = 150 + 100/2 = 200$$

$$\text{centerY} = 150 + 60/2 = 180$$

Now rotate around center:

```
push();
```

```
translate(200, 180); // move origin to center
rotate(PI/4); // 45 degrees
rect(-50, -30, 100, 60); // centered drawing
pop();
```

Why rect(-50, -30, 100, 60) ?

Because when we rotate around the center:

- Half width = 50
- Half height = 30

So we draw the rectangle **centered at (0,0)**:

Top-left corner:

(-50, -30)

Otherwise the rotation will be around the top-left — causing the shape to swing in a circle.

LAB NO#14

Color & Visual Attributes

In computer graphics, color and styling functions allow us to control:

- **Border** (stroke)
- **Inside color** (fill)
- **Thickness** (strokeWeight)
- **Transparency** (alpha)
- **Different color modes** (RGB, HSB, HEX, Grayscale)

These attributes help us create attractive drawings, patterns, and UI graphics.

Stroke (Shape Border)

stroke (r, g, b)

Sets border color.

```
stroke(255, 0, 0); // red border
```

```
rect(50, 50, 100, 60);
```

```
1 function setup() {
2   createCanvas(400, 400);
3   background(220);
4 }
5 function draw() {
6   stroke(255, 0, 0); // red border
7   rect(50, 50, 100, 60);
8 }
9 }
```



strokeWeight (thickness)

Sets border thickness.

```
strokeWeight(5);
```

```
line(50, 50, 200, 50);
```

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4 }  
5 function draw() {  
6   strokeWeight(5);  
7   line(50, 50, 200, 50);  
8 }  
9 }
```



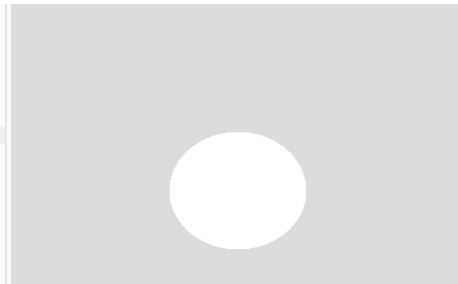
noStroke()

Removes border completely.

```
noStroke();
```

```
circle(200, 200, 120);
```

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4 }  
5 function draw() {  
6   noStroke();  
7   circle(200, 200, 120);  
8 }  
9 }
```



Fill (Inside Color)

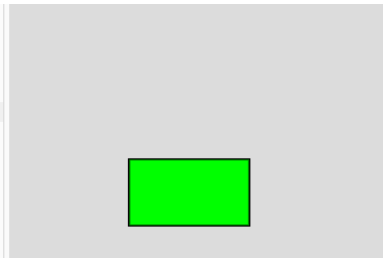
fill(r, g, b)

Sets inside color.

```
fill(0, 255, 0); // green inside
```

```
rect(100, 150, 100, 60);
```

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4 }  
5 function draw() {  
6   fill(0, 255, 0); // green inside  
7   rect(100, 150, 100, 60);  
8 }  
9 }
```

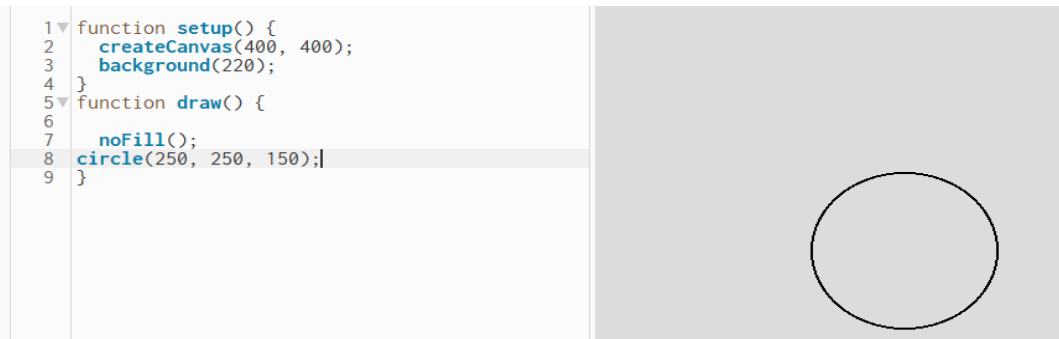


noFill()

Draws only the border, no inside color.

```
noFill();
```

```
circle(250, 250, 150);
```

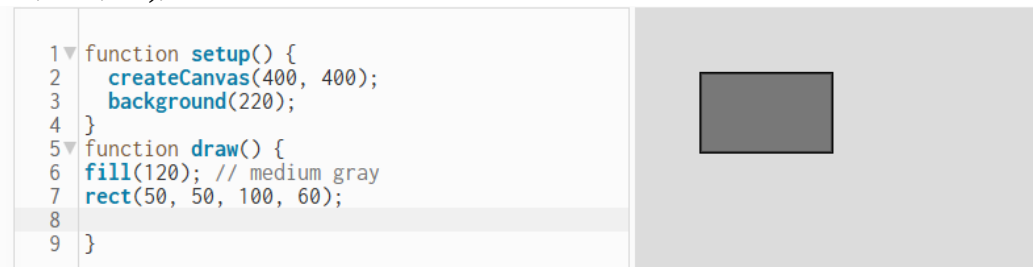
Color Modes in p5.js

p5.js supports five main color formats:

Grayscale

One value only: 0–255

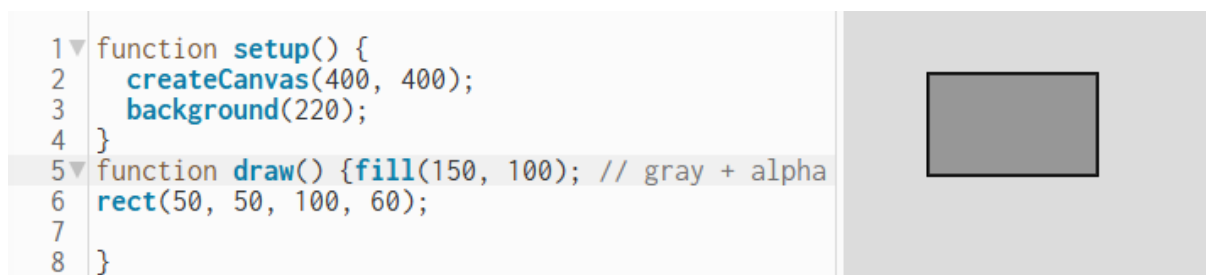
- 0 = black
 - 255 = white
 - Middle values = gray
- `fill(120); // medium gray`
`rect(50, 50, 100, 60);`



Grayscale Transparency Range

Alpha: 0 → 255

`fill(150, 100); // gray + alpha`

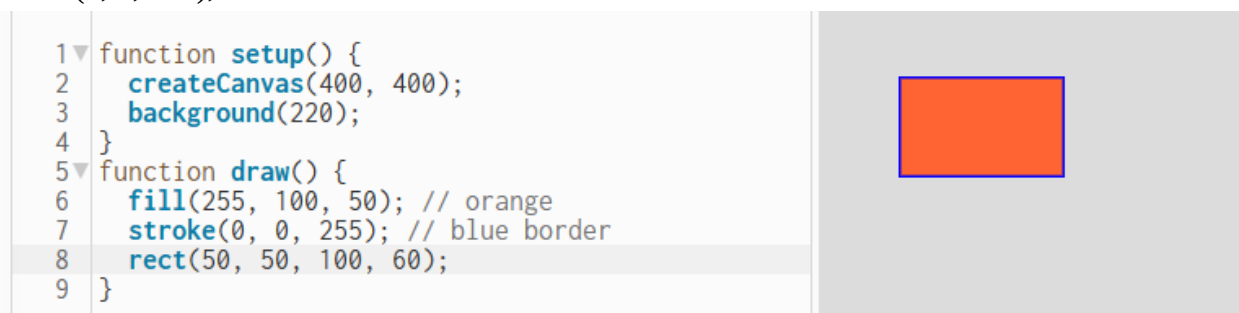


RGB Mode

Standard format using Red, Green, Blue (0–255 each).

`fill(255, 100, 50); // orange`

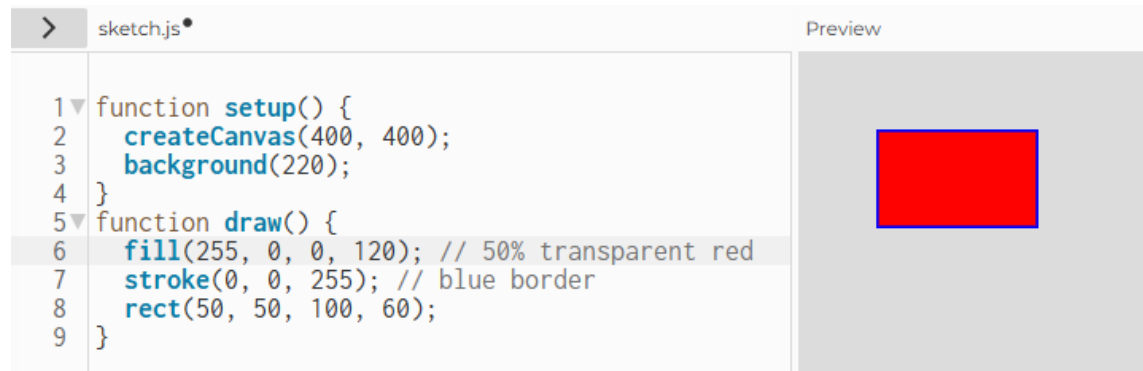
`stroke(0, 0, 255); // blue border`



RGB Transparency Range

Alpha: 0 → 255

`fill(255, 0, 0, 120);` // 50% transparent red



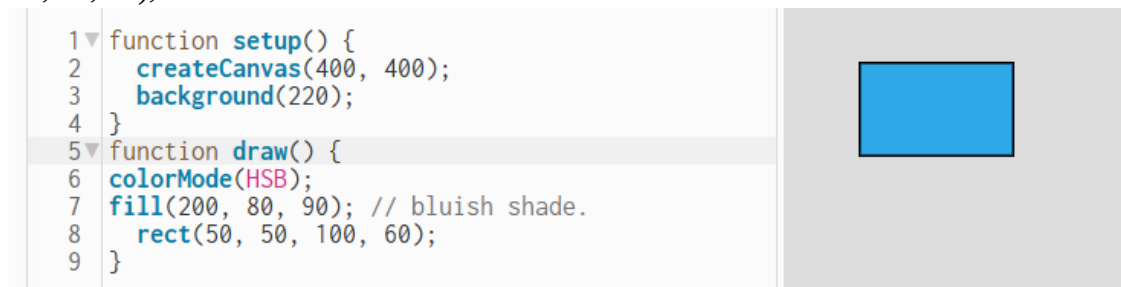
HSB Mode (Hue, Saturation, Brightness)

More intuitive for gradients and patterns.

Enable mode:

`colorMode(HSB);`

`fill(200, 80, 90);` // bluish shade.



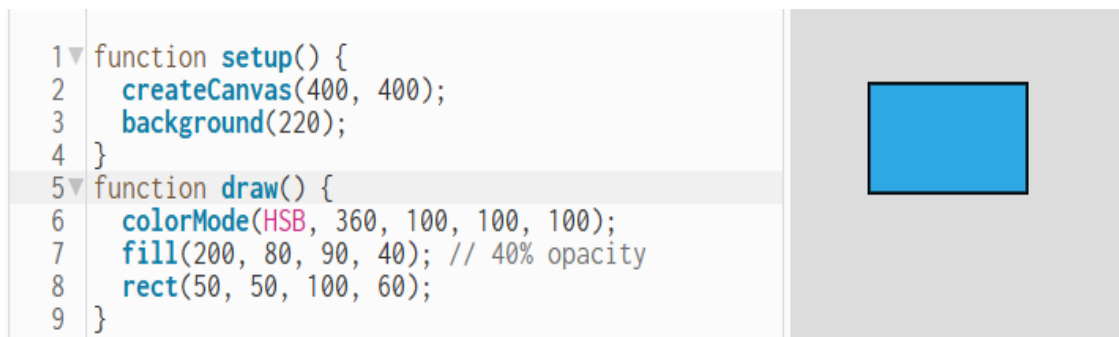
HSB Transparency Range

Alpha: 0 → 100

// IMPORTANT: set HSB ranges properly

`colorMode(HSB, 360, 100, 100, 100);`

`fill(200, 80, 90, 40);` // 40% opacity



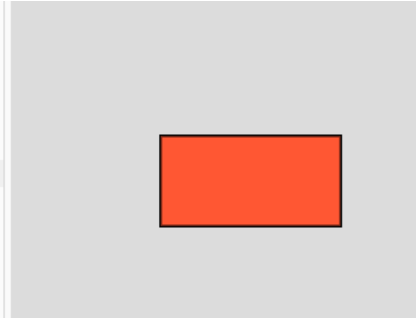
HEX Colors (Web Colors)

Format: #RRGGBB

```
fill("#FF5733"); // orange-red
```

```
rect(100, 100, 120, 60);
```

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4 }  
5 function draw() {  
6   // Format: #RRGGBB  
7   fill("#FF5733"); // orange-red  
8   rect(100, 100, 120, 60);  
9 }  
10 }
```



HEX Transparency

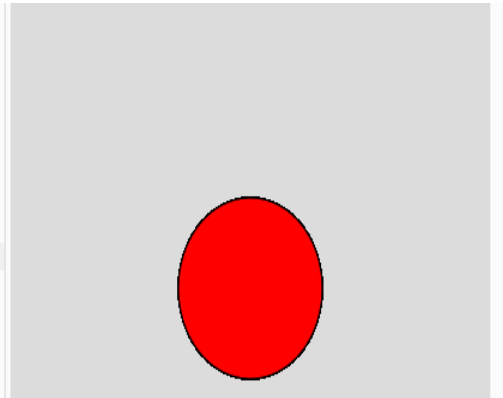
Format: #RRGGBBAA

AA = transparency (00–FF)

```
fill("#FF000080"); // 50% transparent red
```

```
circle(200, 200, 120);
```

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(220);  
4 }  
5 function draw() {  
6   //Format: #RRGGBBAA  
7   //AA = transparency (00–FF)  
8   fill("#FF000080"); // 50% transparent red  
9   circle(200, 200, 120);  
10 }
```

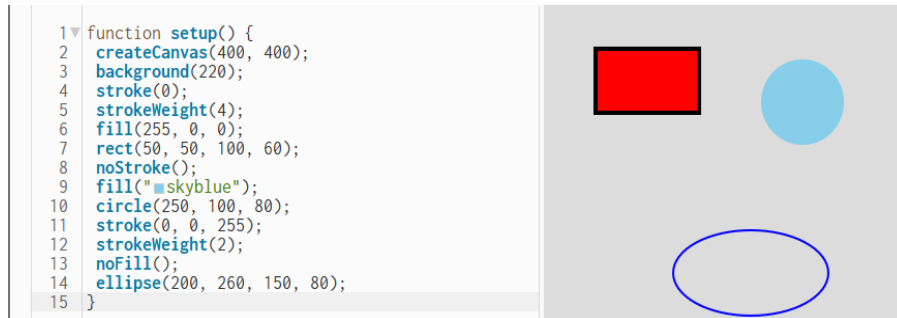


Transparency Summary

Color Mode	Alpha Range
RGB	0 → 255
HSB	0 → 100
HEX	00 → FF
Grayscale	same as RGB (0 → 255)

Example Programs

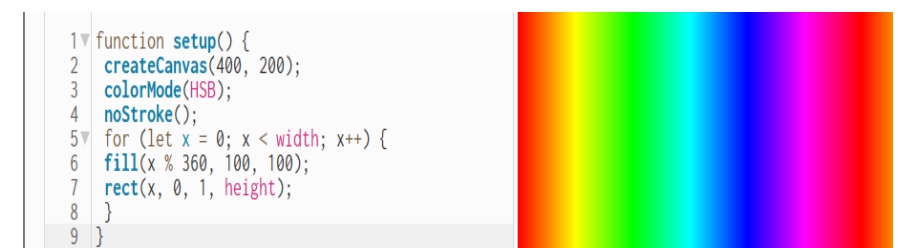
Example 1 – Mixed Shapes with Styling



Example 2 – Pattern with Random Colors



Example 3 – HSB Gradient

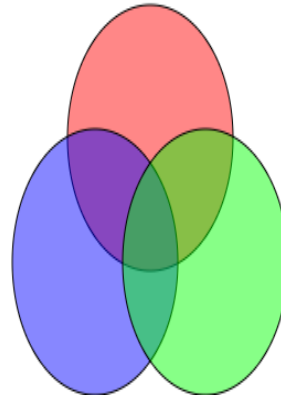


Example 4 – grayscale Gradient



Example 5 – Transparent Overlapping Circles

```
1 function setup() {  
2   createCanvas(400, 400);  
3   background(255);  
4   fill(255, 0, 0, 120);  
5   circle(200, 150, 150);  
6   fill(0, 0, 255, 120);  
7   circle(150, 220, 150);  
8   fill(0, 255, 0, 120);  
9   circle(250, 220, 150);  
10 }
```



Summary for Students

stroke() → border color

strokeWeight() → thickness

fill() → inside color

noFill() → only border

noStroke() → only fill

RGB = 0–255 colors

HSB = hue-based colors

HEX = web colors

Alpha = controls transparency

Grayscale = simplest color form

Color settings stay active until change.

Curve Generation (BÉZIER CURVES)

Introduction to Curves in Computer Graphics

In computer graphics, we often need smooth shapes instead of straight lines. Examples include:

- logos
- icons
- fonts
- UI shapes
- animation paths
- character outlines

To create such smooth shapes, CG uses **curves based on control points**.

In this lab, we focus on the **Bézier Curve**, which is the most used curve in graphics software (Adobe Illustrator, Photoshop, CorelDraw, Figma, etc.).

Bézier Curves (Most Important Curve in CG)

A Bézier curve uses 4 points:

- Start point → P0
- Control point 1 → P1
- Control point 2 → P2
- End point → P3

The curve always:

- Starts at P0
- Ends at P3
- Bends toward P1 and P2
- Does NOT pass through P1 and P2 (usually)

Control points act like magnets — they pull the curve towards them.

Bézier Curve Formula in p5.js

In p5.js, a Bézier curve is drawn using:

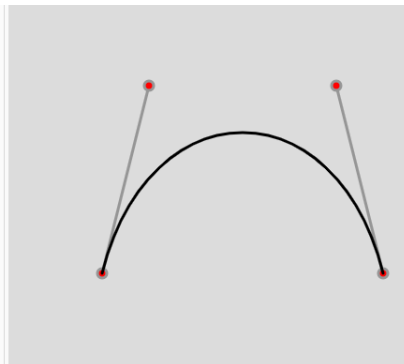
`bezier(x1, y1, cx1, cy1, cx2, cy2, x2, y2);`

Where:

- (x1, y1) = start point
- (cx1, cy1) = control point 1
- (cx2, cy2) = control point 2
- (x2, y2) = end point

Basic Bézier Curve Code (p5.js version)

```
1 function setup() {  
2   createCanvas(500, 400);  
3   background(220);  
4   strokeWeight(3);  
5   // Control points for Bézier curve  
6   let x1 = 100, y1 = 300; // Start point  
7   let cx1 = 150, cy1 = 100; // Control point 1  
8   let cx2 = 350, cy2 = 100; // Control point 2  
9   let x2 = 400, y2 = 300; // End point  
10  // Draw helper lines  
11  stroke(150); // light gray  
12  line(x1, y1, cx1, cy1); // Start → Control1  
13  line(x2, y2, cx2, cy2); // End → Control2  
14  // Draw control points  
15  fill("red");  
16  circle(x1, y1, 10); // Start point  
17  circle(cx1, cy1, 10); // Control point 1  
18  circle(cx2, cy2, 10); // Control point 2  
19  circle(x2, y2, 10); // End point  
20  // Draw Bézier curve  
21  stroke("black");  
22  noFill();  
23  bezier(x1, y1, cx1, cy1, cx2, cy2, x2, y2);  
24 }
```



Explanation of Each Component

Define points We choose 4 points that control the curve shape.

Helper lines

We draw faint lines between:

- start → control point 1
- end → control point 2

These lines help visualize influence on the curve.

Control points drawn as red circles

Easier to understand where the curve bends.

Bézier curve

The curve bends toward control points but touches only start and end.

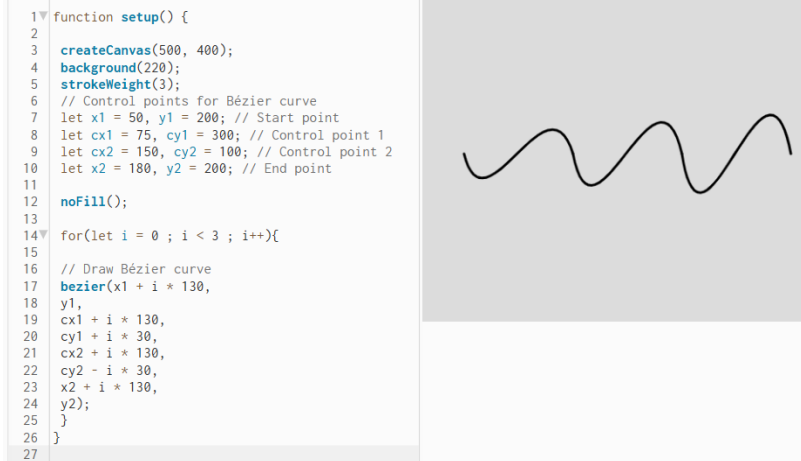
How Bézier Curve Behaves (Important Theory)

- Moving control point 1 change the first half of the curve
- Moving control point 2 changes the second half
- The curve usually does not pass through control points.
- Farther control points create sharper curves
- Closer control points create smoother curves.

Difference Between Line vs Bézier Curve

Feature	Line	Bézier Curve
Definition	2 points	4 points
Shape	Straight	Smooth curved
Control	Only start/end	Control points bend curve
Usage	Basic shapes	Graphics, fonts, icons

Multiple Bézier Curve Code (p5.js version)



Key Takeaways:

- A Bézier curve uses 4 points
- The curve touches only start & end
- Two control points bend the curve
- p5.js gives a simple function → bezier()
- Moving control points changes curve shape
- Used everywhere: fonts, logos, graphics etc.

LAB NO#15

Animation & Motion

What is Animation in Computer Graphics?

Animation is created by showing many images (frames) very quickly. Each frame is slightly different from the previous one. This creates an illusion of motion (just like cartoons, movies, games).

In p5.js, animation is done using the:

draw () loop

draw () runs repeatedly (60 times per second by default). Whatever you change inside draw () creates **movement**.

Important Animation Concepts

Frame Updates:

Every time draw() runs, a new frame is created. Variables store object position.

Example:

$x = x + 2$; → object moves right

$y = y + 3$; → object moves down

Redrawing:

Shapes must be redrawn in every frame.

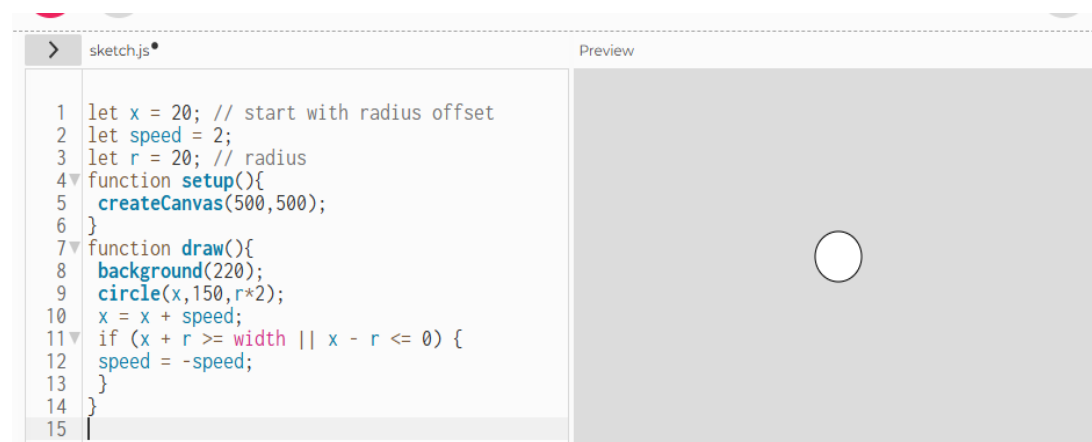
Clearing the screen:

Use background () inside draw () to avoid trails (old drawings).

Step 1: Simple Animation Code



Step 2: Add Bounce (Basic Version)



This program moves the circle from left to right.

We use $x + r$ and $x - r$ because collision is checked from the center of the circle to its edges.

Here, x represents the center of the circle.

Why do we use r (radius)

We don't check $x \geq \text{width}$ and $x \leq 0$ because the circle has size.

So, we check edges using radius:

- Right edge $\rightarrow x + r$
- Left edge $\rightarrow x - r$

Important Concept – Why $r * 2$?

We use `circle(x,150, r*2);` Because:

r = radius

`circle()` needs diameter

If you write:

`circle(x,150, r);`

Then:

Actual radius becomes $r/2$

But collision still uses r

Result: early bounce (wrong)

Rule: Drawing size and collision size must match.

Problem in this version:

- Sometimes causes jerking or vibration at edges
- Can behave incorrectly at high speed
- Does not correct object position after collision.

Step 3: Fix Start Position

Wrong:

`let x = 20;`

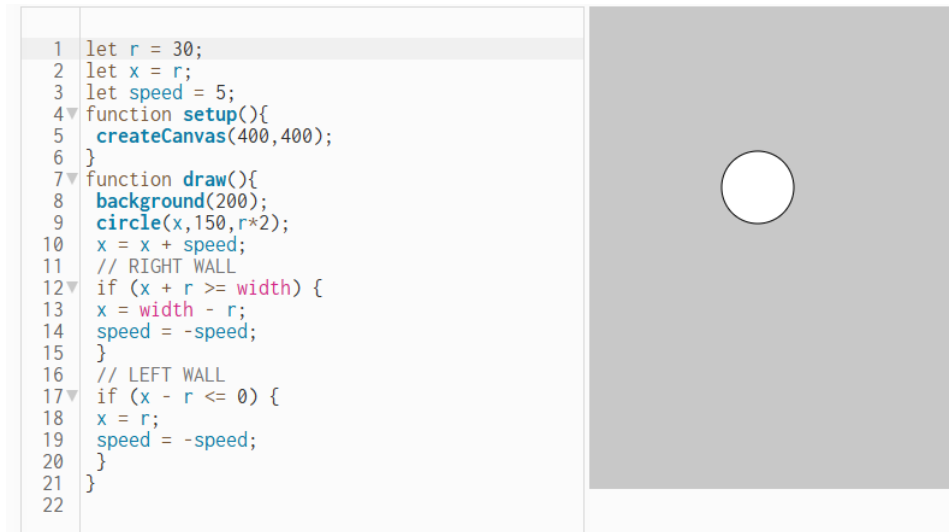
If $r = 30$, object starts outside boundary

Correct:

`let x = r;`

Always start inside the screen.

Step 4: Final Correct Bounce (No Bugs)



This is the recommended approach for real applications and assignments.

Why is this correct:

Moves object

Detects collision

Fixes position

Reverses direction

Works at any speed

Common Mistakes :

Mistake 1: Wrong size

- `circle(x,150,r);`
- Causes early collision

Mistake 2: Wrong start

- `let x = 10;`
- Object starts outside → stuck

Mistake 3: Only reversing speed

- `speed = -speed;`
- Causes vibration

One-Line Summary:

“Move → detect → fix position → reverse direction”

Key Takeaways:

- Animation happens inside draw ()
- Variables store motion
- Change variables → object moves
- Background () clears screen each frame
- Collision requires both detection and position correction.